

Master Degree in Applied and Computational Math
2025–26

Master Thesis

“Parallel variational integrators on forced systems and Lie groups”

Odysseas Machairas

Advisor: David Martín de Diego
University tutor: Luis Alberto Ibort Latre
Madrid, June 2026

AVOID PLAGIARISM

The University uses the **Turnitin Feedback Studio** program within the Aula Global for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

SUMMARY

Variational integrators discretize the configuration space of a Lagrangian mechanical system rather than its equations of motion, making it possible to construct algorithms that preserve the geometric structure of the underlying manifold. Solving the resulting discrete Euler-Lagrange (DEL) equations is typically sequential, but a recent parallel Jacobi-Newton algorithm overcomes this, combining parallel Jacobi updates with local Newton-Raphson steps; its convergence is governed by a block-tridiagonal Hessian matrix. The algorithm is, however, formulated only for unforced systems on vector spaces.

This thesis extends it in two directions, together with the corresponding convergence analysis. The first is to forced (Lagrange-d'Alembert) systems, such as those with dissipation; the method adapts by adding a force term, and converges if a per-edge condition holds, or if sufficiently small forces (relative to the contraction margin of the iteration) are added to a convergent unforced system. The second is to configuration spaces that are Lie groups, such as those arising in rigid body dynamics; here the Newton-Raphson correction is computed in the Lie algebra and applied via a retraction and a pullback, and the convergence criteria, read in a Lie algebra chart at the solution where the retraction's curvature terms vanish, are structurally identical to the vector space case.

Keywords: Variational Integration, Discrete Euler-Lagrange Equations, Parallel Jacobi-Newton Method, Forced Systems, Lie Group Integrators, Convergence Analysis

CONTENTS

1. INTRODUCTION.	1
2. THEORETICAL BACKGROUND	3
2.1. Basic mathematical notions	3
2.2. Physical systems, configuration space and Lagrangians	6
2.2.1. The Euler-Lagrange equations	7
3. NUMERICAL APPROXIMATION TO EULER-LAGRANGE SOLUTIONS	9
3.1. Geometric properties of the Euler-Lagrange equations	9
3.2. Pitfalls of discretizing the equations of motion	10
3.3. Variational integration	11
3.4. Parallel algorithm	13
3.4.1. Convergence criteria	14
4. FORCED SYSTEMS	17
4.1. The parallel algorithm on forced systems	17
4.2. Convergence	18
4.2.1. A local condition	20
4.2.2. A perturbation condition.	22
4.3. Examples	25
5. LIE GROUPS	31
5.1. The parallel algorithm on Lie algebras	31
5.2. Convergence	33
5.2.1. The Hessian in Lie algebra coordinates	34
5.2.2. Convergence criteria	35
5.3. Implementation and examples.	38
6. CONCLUSIONS AND FUTURE WORK	44
6.1. Future work	45
BIBLIOGRAPHY.	47
A. THE LOCAL CONDITION IN THE SCALAR TWO-NODE CASE	
B. DECLARATION OF USE OF GENERATIVE AI	

1. INTRODUCTION

In the Lagrangian formulation of mechanics, possible trajectories of a system in a configuration space ($q(t) \in Q$, where Q is a smooth manifold) are given as the stationary points of the action, defined as $S(q(\cdot)) = \int_0^T L(q(t), \dot{q}(t)) dt$. While it is convenient to work analytically with these equations when possible, in practice the Lagrangian is often either too complicated to handle this way or fundamentally numerical. These cases are intractable to handle analytically, so it is very common to instead find approximations numerically.

In this thesis we give an overview of discretization techniques used to find solutions to Lagrangian systems. Namely, we discuss variational integration, where one discretizes the configuration space and the variational principle itself rather than the equations of motion, which preserves geometric properties of the underlying manifold of the system. We also showcase a state-of-the-art algorithm that can solve regular Lagrangian systems on vector spaces in parallel with fixed boundary conditions [1].

Building on this algorithm, we extend it to work with forced systems and systems defined on Lie groups as configuration spaces. In each case the extension is largely a matter of adapting the existing derivation to the new structure, but it requires establishing the conditions under which the iterative scheme still converges, which do not carry over automatically. This extension makes the algorithm applicable to many more real-life classes of problems. For instance, forced systems give access to dissipation, nonholonomic constraints and externally controlled forces; and Lie groups to rigid body and multi-body dynamics, such as spacecraft attitude control and robotic manipulators.

While we do not provide a foundational and in-depth reference of mathematical and numerical computations of physical systems, this thesis is meant to serve as a motivated introduction to numerical computations of Euler-Lagrange equations. For instance, a programmer with some mathematical background working on a satellite docking system would be able to implement a solver, but could also understand *why* the algorithm works, and even why it has the structure it has in the first place. We believe that having this insight is valuable, especially as mechanical tasks keep getting more automated.

Objectives The objectives of this thesis are:

1. to review variational integration and the parallel iterative solver for the discrete Euler-Lagrange equations of [1];
2. to extend this solver to forced (Lagrange-d'Alembert) systems, together with provable convergence criteria; and

3. to extend it to configuration spaces that are Lie groups, by trivializing to the Lie algebra, and providing convergence criteria.

Each extension is accompanied by a reference implementation and numerical examples (sections 4.3 and 5.3) with the source code publicly available at <https://github.com/sebajf/pdel-mechanics> (GitHub) and <https://git-archive.odilf.com/odilf/pdel-mechanics> (archived snapshot).

Structure The thesis is structured as follows: In chapter 2 we discuss mathematical tools and physical principles that frame the rest of the work. In chapter 3 we explain how to approach discretization and to compute solutions of Lagrangian systems, showcasing specifically the parallel algorithm as presented in [1]. Finally, we explain how to generalize said algorithm to forced systems in chapter 4 and to Lie groups in chapter 5; and give conclusions and avenues for future work in chapter 6.

2. THEORETICAL BACKGROUND

In this chapter we give a brief overview of the mathematical tools and the physical principles used in the thesis. Section 2.1 outlines the main geometrical concepts used that the reader should be familiar with; while section 2.2 is a friendlier, more motivated introduction to modeling of physical systems using Lagrangians.

In this chapter and in the rest of the thesis, whenever a new term is introduced, it is *highlighted like this*.

2.1. Basic mathematical notions

As mentioned above, this section is not meant to be an introduction to the topics, but rather more of a clarification/refreshers. For each mathematical notion listed, specific references to [2] are given, which contain motivated introductions and more in-depth explanations.

Notions 2.1.2–2.1.6 are used often throughout the thesis. Notions 2.1.7–2.1.11, which are annotated with a star, are only invoked when describing certain geometric properties in sections 3.1 and 3.2, and can be glossed over without too much concern.

2.1.1 Notation for derivatives We use $D_i f$ to represent the partial derivative of a function f over its i -th argument, and interpret repeated indices as higher derivatives, $D_{ij} = D_j \circ D_i$ (so $D_{12} L_d$ is a mixed second derivative). Without a subscript, Df denotes the full derivative over all arguments at once. When we differentiate over a named variable instead of a positional argument, we write $\partial_x f$, such as $\partial_{q_k} S_d$; and in the classical continuous setting we also use the Leibniz form $\frac{\partial L}{\partial q}$. Furthermore, $\frac{df}{dt}$ is the derivative over a scalar, $T_p f$ the differential of a map between manifolds (2.1.4), and $d\alpha$ the exterior derivative (2.1.10).

2.1.2 Smooth manifolds [2, §1] An n -dimensional manifold is a set Q with some topology where for every point $q \in Q$ there exists a neighborhood $U \subset Q$ and a homeomorphism $\phi : U \rightarrow \hat{U}$ where $\hat{U} \subset \mathbb{R}^n$. The tuple (U, ϕ) is called a *chart*, and can be thought of as providing *local coordinates* around a point q .

Multiple charts can form an *atlas* if the neighborhoods U_i cover the original set Q . Two charts (U_i, ϕ_i) and (U_j, ϕ_j) are *smoothly compatible* if $\phi_i \circ \phi_j^{-1}$ is smooth on the intersection $U_i \cap U_j$ (where by “smooth” we mean C^∞). A manifold with an atlas composed of smoothly compatible charts is a *smooth manifold* (see also [3] for more details)

On a more intuitive level, smooth manifolds are topological spaces that look locally like $\mathbb{R}^n$ and where doing a change of coordinates (or a change of “basis”) is smooth.

2.1.3 Tangent spaces [2, §3] The tangent space at point $q \in Q$ is denoted as $T_q Q$ and is defined by representatives of:

$$T_q Q = \{\dot{\gamma}(0) \mid \gamma(0) = q\}$$

where $\gamma(t) : (-\varepsilon, \varepsilon) \rightarrow Q$ are differentiable paths on Q . Tangent spaces always form n -dimensional vector spaces and we denote by $\tau_Q : TQ \rightarrow Q$ the canonical projection. These vector spaces have duals, called *cotangent spaces* $T_q^* Q$ that can be interpreted as linear forms $T_q Q \rightarrow \mathbb{R}$. Also, $\pi_Q : T^* Q \rightarrow Q$ denotes the corresponding canonical projection.

The *tangent bundle* is the disjoint union of all tangent spaces, $TQ = \sqcup_q T_q Q$, which forms a $2n$ -dimensional manifold itself (that is not in general a vector space). The *cotangent bundle* is defined in the same way.

For our purposes, a mapping to and from tangent and cotangent bundles such as $f : TM \rightarrow TM$ is said to be *fibre-preserving* if for every $q \in M$ and every $\eta \in T_q M$, $f(\eta) \in T_q M$. In other words, f does not move the point at which it is tangent or $\tau_Q \circ f = \tau_Q$.

2.1.4 Differential [2, p. 55] A differential or derivative of a function on manifolds $f : M \rightarrow N$ evaluated at $p \in M$ is a mapping between the tangent spaces of the manifolds $T_p f : T_p M \rightarrow T_{f(p)} N$, defined as $T_p f(\dot{\gamma}(0)) = \left. \frac{d}{dt} \right|_0 f(\gamma(t))$.

2.1.5 Lie groups [2, §7] A Lie group is a smooth manifold G equipped with a smooth group operation $\cdot : G \times G \rightarrow G$ respecting the usual group axioms (including having a smooth inverse). The tangent space at the identity of a Lie group is called a *Lie algebra*, denoted by $\mathfrak{g}$ (called this way because it forms an algebra with the Lie bracket, although we will not use it in this thesis). We define the left and right translation operators as $L_g = h \mapsto g \cdot h$ and $R_g = h \mapsto h \cdot g$ respectively.

2.1.6 Lie algebra trivialization The group structure of a Lie group allows us to *trivialize* tangent spaces to the Lie algebra. Specifically, the entire tangent bundle TG is isomorphic to $G \times \mathfrak{g}$ via the differential of the left-application, $T_e L_g : \mathfrak{g} \rightarrow T_g G$. In other words, every tangent vector on a Lie group can be described by the point at which it is tangent, and an element of $\mathfrak{g}$.

2.1.7 Differential forms* [2, §14] A *1-form* on a manifold M is a smooth assignment of a covector $\alpha_p \in T_p^* M$ to each $p \in M$ (i.e., a covector field). The differential of a function $Q \rightarrow \mathbb{R}$ is an example of a one-form.

More generally, a *k-form* on M is a smooth assignment of an alternating multilinear map $\omega_p : T_p M \times \cdots \times T_p M \rightarrow \mathbb{R}$ with k arguments for each point $p \in M$. ‘‘Alternating’’ means that swapping any two arguments flips the sign. In particular, a 2-form satisfies

$\omega(v, w) = -\omega(w, v)$. A k -form is often interpreted as providing a signed measure of the infinitesimal parallelepiped spanned by k tangent vectors, with a possibly position-dependent weight.

2.1.8 Wedge product* The *wedge product* of two 1-forms α and β is the 2-form

$$(\alpha \wedge \beta)(v, w) = \alpha(v)\beta(w) - \alpha(w)\beta(v).$$

In coordinates on $\mathbb{R}^2$, $dx \wedge dy$ evaluated on two tangent vectors returns the signed area of the parallelogram they span (i.e., the determinant). The definition extends to higher degree forms as $\alpha \wedge \beta = (-1)^{kl}\beta \wedge \alpha$ for a k -form and an l -form.

2.1.9 Pullback of forms* [2, p. 359] Given a smooth map $f : M \rightarrow N$ and a k -form ω on N , the *pullback* $f^*\omega$ is the k -form on M defined by

$$(f^*\omega)_p(v_1, \dots, v_k) = \omega_{f(p)}(T_p f(v_1), \dots, T_p f(v_k)).$$

That is, the pullback “reverses” the order of operation by pre-applying the differential of f , and then evaluating ω at $f(p)$. Pullbacks respect the wedge product in the sense that $f^*(\alpha \wedge \beta) = f^*\alpha \wedge f^*\beta$.

2.1.10 Exterior derivative* [2, §14] The *exterior derivative* $d : \Omega^k(M) \rightarrow \Omega^{k+1}(M)$ is the unique linear operation that agrees with the ordinary differential on functions ($k = 0$), satisfies the graded Leibniz rule

$$d(\alpha \wedge \beta) = d\alpha \wedge \beta + (-1)^k \alpha \wedge d\beta \quad \text{for } \alpha \text{ a } k\text{-form,}$$

and squares to zero, $d^2 = 0$. A form ω with $d\omega = 0$ is called *closed*.

In coordinates, given $f : U \rightarrow \mathbb{R}$, if $\omega = f dx^{i_1} \wedge \dots \wedge dx^{i_k}$ then $d\omega = df \wedge dx^{i_1} \wedge \dots \wedge dx^{i_k}$. For $\mathbb{R}^3$, applied to a 0-form ($Q \rightarrow \mathbb{R}$) it gives the gradient ∇ , to a 1-form the curl $\nabla \times$, and to a 2-form the divergence $\nabla \cdot$. The identities $\nabla \times \nabla = 0$ and $\nabla \cdot (\nabla \times) = 0$ are both instances of $d^2 = 0$.

2.1.11 Symplectic forms* [2, §22] A *symplectic form* on a manifold M is a 2-form ω that is closed ($d\omega = 0$) and non-degenerate, meaning that $\omega_p(v, \cdot) = 0$ as a 1-form implies $v = 0$. Non-degeneracy forces M to be even-dimensional. A diffeomorphism $F : M \rightarrow M$ is called *symplectic* if it preserves a symplectic form ω , in the sense that $F^*\omega = \omega$.

2.2. Physical systems, configuration space and Lagrangians

A physical system can be framed using either Hamiltonian or Lagrangian mechanics. In this thesis, we focus largely on the Lagrangian approach, where physical systems are described by elements of some *configuration space* Q , which has to be a smooth manifold; and a *Lagrangian* $L : TQ \rightarrow \mathbb{R}$.

Configuration space Even though essentially all physical models try to model reality, which ultimately lives in $\mathbb{R}^3$, the state space need not be $\mathbb{R}^3$ or even a vector space at all. Using manifolds that have some geometric structure can encode constraints directly in the description of the system (such as rigid motions), making it impossible by construction to have physically unrealizable states. That is, the geometry of the manifold can encode constraints about periodicity, topology or other types of global constraints.

For instance, one can try to encode the state of a pendulum with $\mathbb{R}^1$, using the angle as a real number. However, this representation does not take into account the periodicity of the angle, since 0 and 2π are different elements of $\mathbb{R}$, but they represent the same physical state. Mathematically, the more accurate approach is to model the angle as an element of S^1 , which is a smooth manifold that encodes the periodicity. The use of $\mathbb{R}^1$ forces one to interpret the fact that 0 and 2π are the same state outside of the mathematical framework, where mathematical rigor can no longer help. Furthermore, if some result hinges on the periodicity of the pendulum, this again needs to be done “out of band” and it cannot be rigorously, mathematically, justified anymore.

While these problems might seem minor for the pendulum example, they apply similarly to much more complicated systems. As a slightly more involved example, it might be intuitive to model rotations in $\mathbb{R}^3$ using Euler angles (i.e., pitch, yaw and roll), but this leads to gimbal lock and, perhaps more crucially, composition of rotations does not correspond to vector addition, so often most of the work is spent in switching coordinates [4, p. 39]. These problems arise from the fundamental fact that 3-dimensional rotations do not have a vector space structure! Instead, we can use $SO(3)$, the Lie group which correctly encodes these rotations. Statements about rotations on $SO(3)$ are rigorous and unambiguous.

Lagrangian On the other hand, the Lagrangian encodes the *laws* of the physical system by governing how states in Q evolve. Namely, we can talk about *paths* in configuration space $q : [0, T] \rightarrow Q \in C^2([0, T], Q)$ which define the *path space of Q* , $\text{Path}(Q) = C^2([0, T], Q)$. Not all paths are realizable in the physical system, and in fact a path can only occur if it is a critical point of the *action* $S : C^2([0, T], Q) \rightarrow \mathbb{R}$, defined as:

$$S(q) = \int_0^T L(q(t), \dot{q}(t)) dt. \quad (2.1)$$

This is the fundamental idea that underpins every result in this thesis, called *Hamilton's*

principle:

Definition 1 (Hamilton's principle). A path $q : [0, T] \rightarrow Q \in C^2([0, T], Q)$ is the physical trajectory of the Lagrangian system defined by $L : TQ \rightarrow \mathbb{R}$ if and only if q is a critical point of the functional action S .

This is the standard Lagrangian formulation, that describes closed systems. In principle this is enough to model any physical system. In practice, however, it is often more convenient to model a smaller subsystem in isolation, even when that subsystem exchanges energy with its surroundings. There are extensions such as time-dependent and higher-order Lagrangians that follow the same Hamilton's principle but with different preservation properties. A few such extensions include:

- *Time-dependent Lagrangians*, $L : \mathbb{R} \times TQ \rightarrow \mathbb{R}$, depend explicitly on a time parameter. This models a system whose effective laws of physics change over time, typically because it is coupled to an external influence that varies with time. Hamilton's principle applies but energy is no longer conserved.
- *Forced systems* are one of the main topics of this thesis (chapter 4). They have some forcing applied (such as dissipation) and follow the Lagrange-d'Alembert principle (definition 3).
- *Constrained* systems restrict the admissible configurations or velocities to a submanifold of TQ . Classic examples include the rigid links of a robotic arm (holonomic constraints) or rolling contact of a wheel on a surface (nonholonomic constraints).
- *Higher-order* Lagrangians depend on higher derivatives of q , e.g. $L(q, \dot{q}, \ddot{q})$. In principle, the results of this thesis are able to be carried over automatically to Lagrangians of arbitrary order [1, §2.2][5, §2].

2.2.1. The Euler-Lagrange equations

Following Hamilton's principle [3, §1.3], we seek to find the critical points of the action defined by a Lagrangian L . This can be done by taking a variation of the action and equating it to 0. Namely, we consider a one-parameter family of paths $q_\varepsilon(t)$ with $q_0 = q$, whose infinitesimal variation is the vector field along q given by $\delta q = \frac{d}{d\varepsilon} q_\varepsilon|_{\varepsilon=0}$, with fixed endpoints $\delta q(0) = \delta q(T) = 0$. We then define the variation of the action as $\delta S(q) = \frac{d}{d\varepsilon} S(q_\varepsilon)|_{\varepsilon=0}$. Applying this to the action, we get:

$$\begin{aligned}
\delta S(q) &= \delta \int_0^T L(q(t), \dot{q}(t)) dt \\
&= \int_0^T \left(\frac{\partial L}{\partial q} \delta q + \frac{\partial L}{\partial \dot{q}} \delta \dot{q} \right) dt \\
\text{(by parts)} \quad &= \int_0^T \left(\frac{\partial L}{\partial q} \delta q - \frac{d}{dt} \frac{\partial L}{\partial \dot{q}} \delta q \right) dt + \frac{\partial L}{\partial \dot{q}} \underbrace{\delta q \Big|_0^T}_0 \\
&= \int_0^T \left(\frac{\partial L}{\partial q} - \frac{d}{dt} \frac{\partial L}{\partial \dot{q}} \right) \delta q dt
\end{aligned} \tag{2.2}$$

and for this to hold given any variation δq , we need the integrand to be identically 0. This is how we derive the Euler-Lagrange equation:

$$\frac{\partial L}{\partial q} - \frac{d}{dt} \frac{\partial L}{\partial \dot{q}} = 0 \tag{2.3}$$

Computing the relevant derivatives of the Lagrangian provides us with an implicit second-order differential equation with respect to $q(t)$. A path that satisfies this differential equation is a path that can occur in the system. The ODE arising from a Lagrangian system results in what are often called the *equations of motion* for that system.

We can try to solve this ODE analytically. Sometimes, this is possible; but often, it is intractable to do so (either because the Lagrangian is too large, or the Lagrangian is computed based on some empirical numerical data), and hence numerical methods are used instead. In the next section, we discuss the framework to discretize and solve the Euler-Lagrange equations numerically.

3. NUMERICAL APPROXIMATION TO EULER-LAGRANGE SOLUTIONS

Perhaps the most straightforward approach to numerically compute solutions that extremize the action given a Lagrangian is to compute the ODE that results from Euler-Lagrange (2.3), and then use some numerical ODE computation scheme based on the underlying type of manifold. For example, if $Q \cong \mathbb{R}^n$ then we can use Runge-Kutta, or if Q is a Lie group we could use Runge-Kutta-Kaas [6] or similar methods.

However, discretization of the resulting ODE based on $q(t)$ can have undesirable effects due to the loss of the underlying geometric picture. In section 3.1 we explain what these properties are and in section 3.2 we discuss what is lost when one stops working geometrically.

Instead, one can discretize the tangent bundle of the configuration space TQ as $Q \times Q$, and derive discrete analogs of the Euler-Lagrange equations themselves. This approach, called *variational integration* [4, §1.3], will be explained in section 3.3.

3.1. Geometric properties of the Euler-Lagrange equations

The Euler-Lagrange equations can be viewed as a dynamical system on the *phase space* TQ . Given a Lagrangian $L : TQ \rightarrow \mathbb{R}$, the *Lagrangian vector field* $X_L : TQ \rightarrow T(TQ)$ is the second-order vector field whose integral curves (i.e., curves whose velocity matches X_L at every point) are the solutions of the Euler-Lagrange equation (2.3). This has an associated flow, called the *EL (Euler-Lagrange) flow*,

$$F_t : TQ \rightarrow TQ, \quad (q(0), \dot{q}(0)) \mapsto (q(t), \dot{q}(t)),$$

that sends each initial state to the state reached after time t along the corresponding solution. In other words, F_t packages the physical evolution of *every* initial condition simultaneously, rather than one trajectory at a time.

It is this flow that has two distinct sources of exactly conserved quantities, with different origins. The first, Noether's theorem, associates a conserved scalar to each continuous symmetry of L . The second, symplecticity, is a property of the variational structure of (unforced) Lagrangians, which in particular implies the exact preservation of a Hamiltonian function. Both of these constrain long-term behavior in ways that local error estimates often fail to capture. We will briefly explain the implications of each (for an in-depth examination, see [4, §1.2] and [3, §§7, 11]).

Conservation laws and Noether's theorem Suppose L admits a *continuous symmetry*: a family of diffeomorphisms $\Phi_s : Q \rightarrow Q$, with $\Phi_0 = \text{id}$, that leaves L invariant in the

sense that

$$L(\Phi_s(q), T\Phi_s \cdot \dot{q}) = L(q, \dot{q}) \quad \text{for all } s, q, \dot{q}, \quad (3.1)$$

These diffeomorphisms are generated infinitesimally by a vector field X on Q given by

$$X(q) = \left. \frac{d}{ds} \right|_{s=0} \Phi_s(q).$$

Noether's theorem states that, in coordinates, the scalar function on TQ

$$J(q, \dot{q}) = \sum_i \frac{\partial L}{\partial \dot{q}^i} X^i(q) \quad (3.2)$$

is conserved along every solution of the Euler-Lagrange equations [3, §7]. That is, $\frac{dJ}{dt} = 0$.

Well-known conservation laws can be derived in this way: translational symmetry of L corresponds to linear momentum, rotational symmetry to angular momentum, time-translation symmetry (considering time as part of the state space, $Q \times \mathbb{R}$) to energy, etc.

Symplecticity In the Euler-Lagrange derivation, integration by parts (2.2) leaves a boundary term that ultimately evaluates to 0 since the endpoints are fixed. However, we can consider this term on its own. Specifically, this term is a one-form on TQ , called the *Cartan one-form* θ_L , expressed as $\frac{\partial L}{\partial \dot{q}^i} dq^i$ in coordinates. Its exterior derivative

$$\omega_L = d\theta_L$$

is non-degenerate and closed, making it a symplectic form on TQ . The EL flow preserves this symplectic form ω_L , in the sense that $F_t^* \omega_L = \omega_L$ for all t [4, §1.2.3]. This makes the EL flow *symplectic* (with respect to ω_L).

Symplecticity of a flow implies that it is (locally) the flow of some Hamiltonian function H , and that function is exactly preserved along the flow [3]. Additionally, symplectic flow also implies *Liouville's theorem*, which states that the EL flow preserves phase space volume. In particular, no region of phase space contracts or expands, so there are no attractors or sinks in the dynamics.

3.2. Pitfalls of discretizing the equations of motion

Once we have written the Euler-Lagrange equation as a system of ODEs on TQ , the obvious move is to plug it into an off-the-shelf integrator (e.g., Runge-Kutta). While the local error per step is controlled, this approach loses the geometric structure and with it the geometric properties described in section 3.1. A priori, a generic integrator does not preserve momentum maps nor is it symplectic. It is possible to construct, on a case-by-case basis, integrators tailored to conserve simple invariants such as linear or angular momentum [7, §§IV.1–IV.2], but it is hard to make this work for more complicated invariants [7,

Thm. IV.3.3]. Symplectic integrators (that are not specific cases of the variational principle) are rarer still [7, §§VI.4.1, VI.6.3, Thm. IV.6.4]; and algorithms that exactly preserve energy cannot be symplectic for general (non-integrable) systems at fixed step size [8, §3].

Furthermore, if Q is not a vector space, sometimes the updates are done in a bigger ambient vector space that Q can be embedded into. Then, accumulation of local errors can result in values that are not elements of Q at all. For instance, with 3-dimensional rotations, integration of $SO(3)$ matrices in the ambient matrix space $\mathbb{R}^9$ often leads to loss of orthogonality and unit determinant. This can be mitigated to some extent by reprojecting onto the relevant manifold, but that might increase errors and makes convergence harder to analyze.

In short, naive integration produces long-term behavior that is qualitatively unphysical. This is especially problematic for conservation laws, as they might be assumed as invariants in a broader analytical context where a numerical integrator might be used.

3.3. Variational integration

It is possible to discretize the variational principle itself (definition 1), rather than the resulting equations of motion. The discrete flow then inherits, by construction, a discrete analog of each of the geometric properties of section 3.1.

The discrete analog of a continuous path $q : [0, T] \rightarrow Q$ is a sequence of points in Q . The Lagrangian can be adapted to this discrete context into a *discrete Lagrangian* L_d that acts on two consecutive points of a path (instead of one point in TQ):

$$L : TQ \rightarrow \mathbb{R} \implies L_d : Q \times Q \rightarrow \mathbb{R}$$

Then, the discrete action is now composed of sums of these discrete Lagrangians. That is, for a path $\mathbf{q} = \{q_k \in Q\}_{k=0}^N$, the discrete action S_d is:

$$S_d(\mathbf{q}) = \sum_{k=0}^{N-1} L_d(q_k, q_{k+1}) \quad (3.3)$$

Notice that this is a reformulation that can be done with no loss of information. That is, an *exact discrete Lagrangian* L_d^e can be defined for a step size h as

$$L_d^e(q_0, q_1) = \int_0^h L(q_{0,1}(t), \dot{q}_{0,1}(t)) dt \quad (3.4)$$

where $q_{0,1}$ is the path that extremizes the action along all the paths that connect q_0 and q_1 . This way the discrete formulation gives identical results to the continuous one. Of course, finding the path $q_{0,1}$ is as hard as the original continuous problem; so, in practice, the discrete Lagrangian L_d is defined using some quadrature rule that essentially approximates the integral of the exact discrete Lagrangian.

Applying Hamilton's principle to the discrete action (that is, $DS_d = 0$), the *discrete Euler-Lagrange* (DEL) equations can be derived:

$$D_2 L_d(q_{k-1}, q_k) + D_1 L_d(q_k, q_{k+1}) = 0. \quad (\text{DEL})$$

Similarly to Hamilton's principle (definition 1), the DEL equations underpin every result in this thesis.

These equations are well-defined for any manifold, since both $D_2 L_d(q_{k-1}, q_k)$ and $D_1 L_d(q_k, q_{k+1})$ live in $T_{q_k}^* Q$, which forms a vector space with well-defined addition.

Geometric properties of the DEL equations As in the continuous case, the DEL equations (DEL) define a discrete flow $(q_{k-1}, q_k) \mapsto (q_k, q_{k+1})$ on $Q \times Q$. Using this discretization, the properties of section 3.1 are automatically inherited. The boundary term in the variation of the discrete action defines a discrete one-form¹ on $Q \times Q$, whose exterior derivative is a symplectic form ω_{L_d} . The discrete flow preserves ω_{L_d} exactly at every step. This makes the discrete flow symplectic, so it also preserves some modified Hamiltonian $\tilde{H}$ to a high order. For the exact discrete Lagrangian L_d^e , that Hamiltonian is the physical energy H ; for a quadrature-based L_d of order p , it approximates H at order $O(h^p)$. This makes it so that the energy along the numerical trajectory does not oscillate further than $O(h^p)$, so while energy might not be exactly conserved locally, the error is globally bounded.

A discrete analog of Noether's theorem holds as well: any continuous symmetry of L inherited by L_d corresponds to a scalar quantity conserved exactly at every step. This is arguably stronger than the continuous Noether statement, since conservation now holds for the actual numerical trajectory rather than for an ideal one being approximated.

One symmetry that does not get automatically inherited is time-translation symmetry. The discrete Lagrangian $L_d(q_k, q_{k+1})$ implicitly depends on a fixed timestep h , and fixing h removes the time variable from the discrete dynamics, so there is no variable left for time-translation to act on. This is why earlier we discussed energy oscillating around the true value instead of being preserved.

Given this approach to discretization, a trajectory can be computed point-by-point, computing q_{k+1} from q_k and q_{k-1} , given two initial datapoints. This is theoretically sound (assuming existence and uniqueness conditions) and a reasonable approach, amounting essentially to forward integration in time. However, if an endpoint is needed, this approach fundamentally does not work [1, p. 6]. The algorithm presented in the next section, which we study for the rest of the thesis, provides solutions to boundary value problems, and does so in parallel.

¹Technically, there are *two* one-forms, θ^+ and θ^- , but both of these define the same symplectic two-form via the exterior derivative [4, p. 15].

3.4. Parallel algorithm

In [1], an iterative algorithm that can solve the discrete Euler-Lagrange equations is proposed. The idea of the parallel algorithm is to treat the (discrete) action as a function of which we want to find roots. I.e., given a path $\mathbf{q} = \{q_k\}_{k=0}^N \in \mathcal{Q}^{N+1}$ and a discrete Lagrangian L_d , we define its (global) residual R as

$$R(\mathbf{q}) = \bigoplus_{k=1}^{N-1} (D_2 L_d(q_{k-1}, q_k) + D_1 L_d(q_k, q_{k+1})) \quad (3.5)$$

(where $\bigoplus$ stacks the residuals into a vector in $\bigoplus_k T_{q_k}^* \mathcal{Q}$), and we want to find a path $\mathbf{q}^*$ such that $R(\mathbf{q}^*) = 0$ where q_0 and q_N are fixed.

At a high level, two schemes are used to find this root. First, we frame the algorithm as a block Jacobi update step that can be done in parallel. However, the Jacobi update is still not directly computable, so we use one or more Newton-Raphson steps to compute the Jacobi update.

Jacobi. The Jacobi update step consists of finding a new path $\bar{\mathbf{q}} = \{\bar{q}_k\}_{k=0}^N$ that makes each summand of the residual 0 with respect to the previous path. That is, we define the *local residual* as

$$r_k(\mathbf{q}, \bar{q}) = D_2 L_d(q_{k-1}, \bar{q}) + D_1 L_d(\bar{q}, q_{k+1}) \quad (3.6)$$

which is a term of the sum, replacing the central q_k by some new value $\bar{q}$. An updated path $\bar{\mathbf{q}}$ where for all inner points k , $r_k(\mathbf{q}, \bar{q}_k) = 0$, is not generally a solution, but under certain conditions (discussed in section 3.4.1), iterating this update will converge to a solution. Crucially, the local residual r_k only involves q_k and its two neighbors q_{k-1} and q_{k+1} (see fig. 3.1), so all interior points can be updated independently and in parallel.

Newton-Raphson. Since the Lagrangian can be arbitrarily complex, the Jacobi step does not have a general way to be computed. Specific kinds of Lagrangians, such as quadratic or cubic ones, have clear closed-form solutions. In lieu of knowing if the Lagrangian has one of these forms ahead of time, we use the Newton-Raphson method to solve a linearization of the problem. So, instead of considering the residual $r_k(\mathbf{q}, \bar{q})$, we consider the linearization ρ_k around q_k :

$$\rho_k(\mathbf{q}, \bar{q}) = r_k(\mathbf{q}, q_k) + D_2 r_k(\mathbf{q}, q_k)(\bar{q} - q_k)$$

Note that this is the step where we apply the hypothesis that $\mathcal{Q}$ is a vector space, since we are doing a linear combination based on the increment.

Finding the root is easy, amounting to solving the linear system

$$\begin{aligned} r_k(\mathbf{q}, q_k) + D_2 r_k(\mathbf{q}, q_k)(\bar{q} - q_k) &= 0 \\ \bar{q} - q_k &= (D_2 r_k(\mathbf{q}, q_k))^{-1} (-r_k(\mathbf{q}, q_k)) \end{aligned}$$

from which we get the general Jacobi-Newton update rule:

$$\bar{q} = q_k - (D_2 r_k(\mathbf{q}, q_k))^{-1} r_k(\mathbf{q}, q_k). \quad (3.7)$$

In a standard discrete Lagrangian system, this derivative of the residual is given by the Hessian of the Lagrangian:

$$\begin{aligned} D_2 r_k(\mathbf{q}, q_k) &= D_{22} L_d(q_{k-1}, q_k) + D_{11} L_d(q_k, q_{k+1}) \\ r_k(\mathbf{q}, q_k) &= D_2 L_d(q_{k-1}, q_k) + D_1 L_d(q_k, q_{k+1}) \end{aligned}$$

It is equation (3.7) which is computable using any linear solver. The algorithm consists of repeating this step until we converge to a solution. Of course, the question now is to verify under what conditions this algorithm does, in fact, converge.

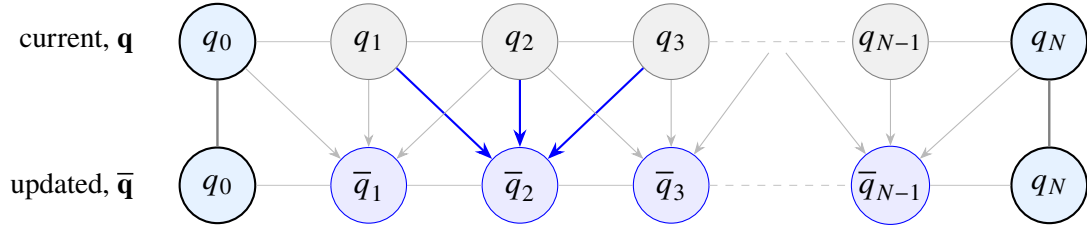


Fig. 3.1. Update scheme of parallel Jacobi-Newton method. Each point in the path is updated using data from the initial position itself and its direct neighbors. All updates are done in parallel, endpoints are kept fixed.

3.4.1. Convergence criteria

The convergence of the method (both in the standard case as well in the forced and Lie group cases) is mainly related to the Hessian of the action, which we refer to as the *global Hessian matrix*, H . This is because, given a problem in the form $\nabla F = 0$, the Jacobi-Newton method converges locally if the Hessian of F is positive definite [1, Prop. 4.5]. Hamilton's principle is precisely such a formulation: the residual that we want to eliminate is the gradient of the action, $R = \nabla S_d$.

Given the structure of the action, its Hessian can be written in terms of second-order derivatives of the Lagrangian. That is, given the action defined as in (3.3) for a path $\mathbf{q}$ of length N

$$\begin{aligned} S_d : \mathcal{Q}^{N+1} &\rightarrow \mathbb{R} \\ S_d(\mathbf{q}) &= \sum_{k=0}^{N-1} L_d(q_k, q_{k+1}) \end{aligned}$$

the derivative with respect to the i -th component is, as usual,

$$\begin{aligned}\partial_{q_i} S_d(\mathbf{q}) &= \partial_{q_i} \sum_{k=0}^{N-1} L_d(q_k, q_{k+1}) \\ &= D_1 L_d(q_i, q_{i+1}) + D_2 L_d(q_{i-1}, q_i)\end{aligned}$$

and then the entries of the Hessian are

$$\begin{aligned}(H(\mathbf{q}))_{ij} &= \partial_{q_i} \partial_{q_j} S_d(\mathbf{q}) = \partial_{q_j} (D_1 L_d(q_i, q_{i+1}) + D_2 L_d(q_{i-1}, q_i)) \\ &= \delta_{i,j} (D_{11} L_d(q_i, q_{i+1}) + D_{22} L_d(q_{i-1}, q_i)) \\ &\quad + \delta_{i+1,j} D_{12} L_d(q_i, q_{i+1}) \\ &\quad + \delta_{i-1,j} D_{21} L_d(q_{i-1}, q_i)\end{aligned}$$

This results in a tridiagonal matrix that can be written in the following form:

$$H(\mathbf{q}) = \begin{pmatrix} \mathcal{D}_1 & C_1 & & & \\ C_1^T & \mathcal{D}_2 & C_2 & & \\ & \ddots & \ddots & \ddots & \\ & & C_{N-3}^T & \mathcal{D}_{N-2} & C_{N-2} \\ & & & C_{N-2}^T & \mathcal{D}_{N-1} \end{pmatrix}. \quad (3.8)$$

where

$$\begin{aligned}\mathcal{D}_k &= D_{22} L_d(q_{k-1}, q_k) + D_{11} L_d(q_k, q_{k+1}) \\ C_k &= D_{12} L_d(q_k, q_{k+1})\end{aligned} \quad (3.9)$$

where by Schwarz's theorem, $C_k^T = D_{21} L_d(q_k, q_{k+1})$.

Thus, the following theorem gives one of the fundamental criteria for convergence of the parallel algorithm ([1, Prop. 4.3, 4.5, remark 4.6])

Theorem 1 (Jacobi-Newton convergence criterion). *If the Hessian $H(\mathbf{q})$ is positive definite, there is a neighborhood $\mathcal{D}$ around a solution $\mathbf{q}^*$ such that any path $\mathbf{q} \in \mathcal{D}$ converges to $\mathbf{q}^*$.*

This condition is sufficient, although in practice it is hard to compute, since it involves checking for positive-definiteness of an $(N-1)n \times (N-1)n$ matrix. The most common way to do this involves the Cholesky factorization, which has an $\mathcal{O}(N^3 n^3)$ runtime.

An easier-to-check condition can be given in terms of a *local Hessian matrix* H_k , which is obtained by decomposing the blocks of the global Hessian matrix as shown in fig. 3.2. The entries of the local Hessian matrix are the following:

$$H_k(q_k, q_{k+1}) = \begin{pmatrix} \mathcal{A}_k & C_k \\ C_k^T & \mathcal{B}_k \end{pmatrix} \quad (3.10)$$

where

$$\begin{aligned}\mathcal{A}_k &= D_{11} L_d(q_k, q_{k+1}) \\ \mathcal{B}_k &= D_{22} L_d(q_k, q_{k+1})\end{aligned} \quad (3.11)$$

(note that $\mathcal{D}_k = \mathcal{B}_{k-1} + \mathcal{A}_k$).

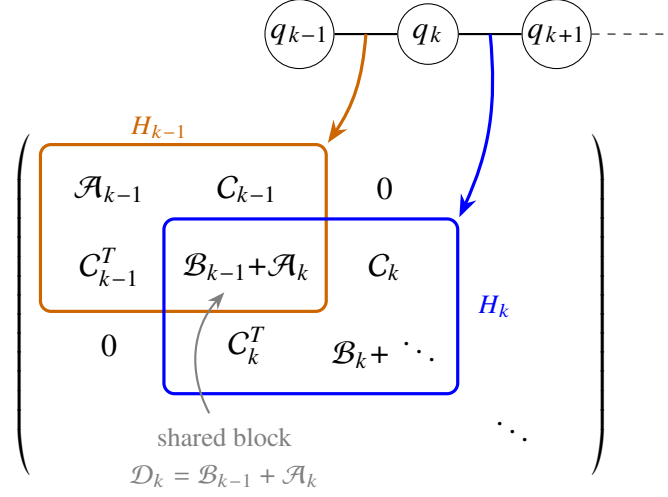


Fig. 3.2. Diagram showing how to assemble the global Hessian from the local Hessians. Each local Hessian sub-block corresponds to one edge of the trajectory.

Theorem 2 (Local Hessian convergence [1, Thm. 4.1]). *If all the local Hessian matrices H_k (as defined above) are positive definite, then the global Hessian matrix H is positive definite and the method converges locally.*

Checking this condition has an $O(Nn^3)$ runtime in terms of the size N of the path and dimension n of the configuration space. However, this condition is not necessary, and in practice it is common for p.d. global Hessians to not be locally p.d..

This is where a third convergence result comes into play, which gives an easy-to-check sufficient *and necessary* condition for the positive definiteness of the global Hessian matrix:

Theorem 3 (Hessian positive definiteness criterion ([1, Thm. 4.12])). *The Hessian matrix $H(\mathbf{q})$ is positive definite if and only if the matrices defined iteratively by $\Lambda_1 = \mathcal{D}_1$ and*

$$\Lambda_i = \mathcal{D}_i - C_{i-1}^T \Lambda_{i-1}^{-1} C_{i-1}, \quad 2 \leq i \leq N - 1$$

are all positive definite.

This check has a runtime complexity of $O(Nn^3)$, which is the same in terms of N as theorem 2, except that this check is a bi-implication.

In the next two chapters, we extend the algorithm to forced systems and Lie groups, and give results analogous to these three theorems when possible.

4. FORCED SYSTEMS

Lagrangian systems can be extended to *Lagrangian forced systems*, defined with a *Lagrangian external force* which is usually defined as a map $f^L : TQ \rightarrow T^*Q$ [9, §4.1.1] that is fibre-preserving. In this setting it is not Hamilton's principle that applies, but rather its forced analog, called the *Lagrange-d'Alembert principle*:

Definition 2 (Lagrange-d'Alembert principle [9, Def. 4.1.1]). A path $q : [0, T] \rightarrow Q \in C^2([0, T], Q)$ is the physical trajectory of the forced Lagrangian system defined by $L : TQ \rightarrow \mathbb{R}$ and force $f^L : TQ \rightarrow T^*Q$ if and only if

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{q}} - \frac{\partial L}{\partial q} = f^L(q(t), \dot{q}(t))$$

Unlike Hamilton's principle, this is *not* in general the extremization of any action.

As in the unforced case, it is possible to carry out a discretization of the state space, the Lagrangian and the forces. The first two are the same: from TQ to $Q \times Q$ and from $L : TQ \rightarrow \mathbb{R}$ to $L_d : Q \times Q \rightarrow \mathbb{R}$.

The discretization of the force is similar to the one of the Lagrangian, except that the force $f : TQ \rightarrow T^*Q$ is mapped to a *pair* of discrete forces $f_d^\pm : Q \times Q \rightarrow T^*Q$ (where f^- is fibre-preserving with respect to the first argument, that is, $\text{pr}_1 = \pi_Q \circ f_d^-$; and f^+ with the second, $\text{pr}_2 = \pi_Q \circ f_d^+$) [4, §3.2.1].

Definition 3 (Discrete Lagrange-d'Alembert principle [9, Def. 4.2.2]). A path $\mathbf{q}$ is physical if and only if, for all displacements $\{\delta q_k\}_{k=1}^{N-1}$ with $\delta q_0 = \delta q_N = 0$,

$$\sum_{k=1}^{N-1} \left(D_2 L_d(q_{k-1}, q_k) + f_d^+(q_{k-1}, q_k) + D_1 L_d(q_k, q_{k+1}) + f_d^-(q_k, q_{k+1}) \right) \cdot \delta q_k = 0.$$

This discretization preserves the relevant geometric properties the same way that is explained in sections 3.1 and 3.3. Solutions that satisfy the discrete Lagrange-d'Alembert principle are given by the forced analog of **(DEL)**, the *forced discrete Euler-Lagrange equations*:

$$D_2 L_d(q_{k-1}, q_k) + f_d^+(q_{k-1}, q_k) + D_1 L_d(q_k, q_{k+1}) + f_d^-(q_k, q_{k+1}) = 0 \quad (\text{FDEL})$$

4.1. The parallel algorithm on forced systems

Extending the parallel algorithm (section 3.4) to use these forces comes down to just modifying the expression of the residual by inserting **(FDEL)** instead of **(DEL)**. Hence, the global residual is modified from **(3.5)** to

$$\hat{R}(\mathbf{q}) = \bigoplus_{k=1}^{N-1} (D_2 L_d(q_{k-1}, q_k) + f_d^+(q_{k-1}, q_k) + D_1 L_d(q_k, q_{k+1}) + f_d^-(q_k, q_{k+1}))$$

and the local residual, from eq. (3.6) to

$$\hat{r}_k(\mathbf{q}, \bar{q}) = D_2 L_d(q_{k-1}, \bar{q}) + f_d^+(q_{k-1}, \bar{q}) + D_1 L_d(\bar{q}, q_{k+1}) + f_d^-(\bar{q}, q_{k+1}).$$

Utilizing these definitions for the residual, and supposing that Q is a vector space, the computation based on the parallel algorithm is the exact same (which is, incidentally, why we introduced the method based on residuals in section 3.4). In fact, it is easy to see that the unforced case already known is recovered by setting the forces to 0 in the forced case.

4.2. Convergence

Figuring out the convergence criteria is trickier. A natural way to proceed is to try to make an argument similar to how [1, Prop. 4.5] is proved. In short, they pose the problem as an extremization $f = \nabla F$, they calculate the Hessian $H = \nabla^2 F$, and if H is p.d. then the algorithm converges. However, this approach fundamentally breaks down because, as mentioned previously, the discrete Lagrange-d'Alembert principle is *not* necessarily an extremization of some F .

Remark 1 (Conservative forces). It is easy to find convergence criteria for a specific family of forces, called *conservative forces*, which *can* be written in terms of a *discrete potential* $V_d : Q \times Q \rightarrow \mathbb{R}$ as:

$$f_d^+(q_k, q_{k+1}) = -D_2 V_d(q_k, q_{k+1}) \quad f_d^-(q_k, q_{k+1}) = -D_1 V_d(q_k, q_{k+1}).$$

This is because this is precisely the case where the Lagrange-d'Alembert principle *is* an extremization. In fact, any system with Lagrangian L_d with forces that admit a discrete potential V_d is identical to an unforced system with Lagrangian $L'_d = L_d - V_d$. In other words, for conservative forces, every theorem of [1] applies trivially. In the rest of the discussion we focus on non-conservative forces.

For generic forces, the relevant criterion that can be applied is given in [1, Lem. 4.4] which we restate here:

Lemma 4 (Convergence of 1-step Jacobi-Newton). *For a general smooth system $f(x) = 0$ (not necessarily a gradient system), the 1-step block Jacobi-Newton method converges locally to x^* if the diagonal blocks of $Df(x^*)$ are regular and $\rho(J(x^*)) < 1$, where $J = -D^{-1}C$, and D and C are the diagonal and off-diagonal parts of Df respectively.*

In practice, to determine whether a specific system converges, there might be no better method than directly estimating $\rho(J(\mathbf{q}^*))$ by power iteration (i.e., Arnoldi/Lanczos). This requires factoring each diagonal block of $Df(\mathbf{q}^*)$ once (an $O(Nn^3)$ setup), then performing block-tridiagonal matrix-vector products at $O(Nn^2)$ per power-iteration step until the dominant eigenvalue stabilizes. If all one wants is a yes/no answer for a concrete problem, this is the most practical method.

In the remainder of this section we aim to provide *structural* sufficient conditions: closed-form criteria stated in terms of the discrete Lagrangian and forces themselves, rather than the assembled iteration matrix. These are valuable for several reasons: they let us prove convergence for entire classes of systems at once (rather than path by path), they generalize the unforced criterion (theorem 2) and make the role of the forces in convergence explicit, and they connect to standard structural conditions in numerical analysis (Gershgorin-style diagonal dominance). With this in mind, we proceed.

To get an expression for J^f , we start by computing the derivative of **(FDEL)** which we will name H^f as it is the forced analog of H in [1] (even though H^f is not the Hessian of any function). The result is the following (similar to (3.8)):

$$\begin{aligned} H_{ij}^f(\mathbf{q}) &= (\mathbf{D}\hat{R}(\mathbf{q}))_{ij} \\ &= \partial_{q_k} (\mathbf{D}_2 L_d(q_{k-1}, q_k) + f_d^+(q_{k-1}, q_k) + \mathbf{D}_1 L_d(q_k, q_{k+1}) + f_d^-(q_k, q_{k+1})) \\ &= \delta_{i,j} (\mathbf{D}_{22} L_d(q_{i-1}, q_i) + \mathbf{D}_2 f_d^+(q_{i-1}, q_i) + \mathbf{D}_{11} L_d(q_i, q_{i+1}) + \mathbf{D}_1 f_d^-(q_i, q_{i+1})) \\ &\quad + \delta_{i+1,j} (\mathbf{D}_{12} L_d(q_i, q_{i+1}) + \mathbf{D}_2 f_d^-(q_i, q_{i+1})) \\ &\quad + \delta_{i-1,j} (\mathbf{D}_{21} L_d(q_{i-1}, q_i) + \mathbf{D}_1 f_d^+(q_{i-1}, q_i)) . \end{aligned}$$

This again has a tridiagonal structure, but is not (necessarily) symmetric. In (3.9) we only had to define the diagonal and the off-diagonal matrices, but now we need three matrices

$$\begin{aligned} \hat{\mathcal{D}}_k &= \mathbf{D}_{22} L_d(q_{k-1}, q_k) + \mathbf{D}_2 f_d^+(q_{k-1}, q_k) + \mathbf{D}_{11} L_d(q_k, q_{k+1}) + \mathbf{D}_1 f_d^-(q_k, q_{k+1}) \\ \hat{\mathcal{C}}_k^+ &= \mathbf{D}_{12} L_d(q_k, q_{k+1}) + \mathbf{D}_2 f_d^-(q_k, q_{k+1}) \\ \hat{\mathcal{C}}_k^- &= \mathbf{D}_{21} L_d(q_k, q_{k+1}) + \mathbf{D}_1 f_d^+(q_k, q_{k+1}) \end{aligned}$$

The resulting global Hessian matrix is, then:

$$H^f(\mathbf{q}) = \begin{pmatrix} \hat{\mathcal{D}}_1 & \hat{\mathcal{C}}_1^+ & & & \\ \hat{\mathcal{C}}_1^- & \hat{\mathcal{D}}_2 & \hat{\mathcal{C}}_2^+ & & \\ & \ddots & \ddots & \ddots & \\ & & \hat{\mathcal{C}}_{N-3}^- & \hat{\mathcal{D}}_{N-2} & \hat{\mathcal{C}}_{N-2}^+ \\ & & & \hat{\mathcal{C}}_{N-2}^- & \hat{\mathcal{D}}_{N-1} \end{pmatrix} \quad (4.1)$$

We also define auxiliary matrices, similar to (3.11):

$$\begin{aligned} \hat{\mathcal{A}}_k &= \mathbf{D}_{11} L_d(q_k, q_{k+1}) + \mathbf{D}_1 f_d^-(q_k, q_{k+1}) \\ \hat{\mathcal{B}}_k &= \mathbf{D}_{22} L_d(q_k, q_{k+1}) + \mathbf{D}_2 f_d^+(q_k, q_{k+1}) \end{aligned} \quad (4.2)$$

(where we still have that $\hat{\mathcal{D}}_k = \hat{\mathcal{B}}_{k-1} + \hat{\mathcal{A}}_k$).

There are two ways to proceed from here. On one hand, we can try to find some criteria of the forced system directly that imply convergence; or we can search for some criteria on the type of forces that, when added to a converged unforced Lagrangian, would also converge with these forces. In the next two subsections we develop both approaches respectively.

4.2.1. A local condition

A natural way to try to extend a result for symmetric to asymmetric matrices is to split them in their *symmetric* and *skew* components. Namely, any matrix M can be written as $M = \text{Sym}(M) + \text{Skew}(M)$, where $\text{Sym}(M) = \frac{1}{2}(M + M^T)$ is symmetric and $\text{Skew}(M) = \frac{1}{2}(M - M^T)$ is skew.

One might expect that an analog of theorem 1 (that the method converges if H is p.d.) might be that the method converges in the forced case if $\text{Sym}(H^f)$ is p.d.. However, this falls short of giving $\rho(J^f) < 1$ on its own (one of the conditions of lemma 4). A minimal counter-example is $M = \begin{pmatrix} 1 & a \\ -a & 1 \end{pmatrix}$ with $D = I$, since then $\text{Sym}(M) = I \succ 0$ for every a , but $J = I - D^{-1}M$ has $\rho(J) = |a|$, which is unbounded.

At this point it is worth getting a mental model of what each block represents. Near a solution $\mathbf{q}^*$ the residual vanishes, so to first order the k th component of the global residual reads

$$R_k^f = \hat{\mathcal{D}}_k e_k + \hat{\mathcal{C}}_{k-1}^- e_{k-1} + \hat{\mathcal{C}}_k^+ e_{k+1},$$

where $e_k = q_k - q_k^*$ is the error at node k . The diagonal block $\hat{\mathcal{D}}_k$ captures how the residual at node k depends on q_k , while the off-diagonal blocks $\hat{\mathcal{C}}_k^+$ and $\hat{\mathcal{C}}_{k-1}^-$ capture how it depends on its two neighbors. The Jacobi-Newton update at node k zeroes *its own* residual while holding the neighbors fixed, that is, it solves $\hat{\mathcal{D}}_k e_k^{\text{new}} = -(\hat{\mathcal{C}}_{k-1}^- e_{k-1} + \hat{\mathcal{C}}_k^+ e_{k+1})$, so

$$e_k^{\text{new}} = -\hat{\mathcal{D}}_k^{-1} (\hat{\mathcal{C}}_{k-1}^- e_{k-1} + \hat{\mathcal{C}}_k^+ e_{k+1}).$$

The new error at k is thus the neighbor errors fed through $-\hat{\mathcal{D}}_k^{-1} \hat{\mathcal{C}}^\pm$, which is exactly one block-row of $J^f = -D^{-1}C$. Note that $\hat{\mathcal{D}}_k$ is itself assembled from the two edges of node k (recall, $\hat{\mathcal{D}}_k = \hat{\mathcal{B}}_{k-1} + \hat{\mathcal{A}}_k$).

Crucially, when $\hat{\mathcal{D}}_k$ is “large enough” to dominate $\hat{\mathcal{C}}^\pm$, these gains have norm < 1 : $\hat{\mathcal{D}}_k^{-1}$ attenuates the incoming error, the global error contracts at every step, and $\rho(J^f) < 1$.

Of course, the question is how to make “large enough” precise. An approach that seems like it would work is to ask whether $\lambda_{\min}(\text{Sym}(H^f)) > \|\text{Skew}(H^f)\|_2$ (this confines the eigenvalues of H^f to a 45° sector in the right half-plane and is another natural generalization of $M \succ 0$ to a non-symmetric setting). This is provable in the scalar $n = 1$, $N = 3$ case (see appendix A) but open in the general block case. What we can prove is a stronger condition *per node*: ask the symmetric part of each diagonal sub-block of H_k^f to strictly dominate the corresponding cross sub-block in spectral norm. This way, we arrive at the following theorem.

Theorem 5 (Local edge dominance implies convergence). *Suppose that, at $\mathbf{q}^*$, for $k = 1, 2, \dots, N-1$,*

$$\lambda_{\min}(\text{Sym}(\hat{\mathcal{D}}_k)) > \|\hat{\mathcal{C}}_k^+\|_2 + \|\hat{\mathcal{C}}_{k-1}^-\|_2, \quad (4.3)$$

with the boundary convention $\hat{\mathcal{C}}_{N-1}^+ = \hat{\mathcal{C}}_0^- = 0$ (these blocks do not appear in H^f). Then each diagonal block $\hat{\mathcal{D}}_j$ is invertible, $\rho(J^f(\mathbf{q}^)) < 1$, and the forced Jacobi-Newton method converges locally to $\mathbf{q}^*$.*

Proof. By hypothesis:

$$\lambda_{\min}(\text{Sym}(\hat{\mathcal{D}}_j)) > \|\hat{\mathcal{C}}_j^+\|_2 + \|\hat{\mathcal{C}}_{j-1}^-\|_2 \geq 0,$$

so $\text{Sym}(\hat{\mathcal{D}}_j) \succ 0$. Then, for any $v \neq 0$, Cauchy-Schwarz gives $\|\hat{\mathcal{D}}_j v\|_2 \|v\|_2 \geq v^T \hat{\mathcal{D}}_j v = v^T \text{Sym}(\hat{\mathcal{D}}_j) v \geq \lambda_{\min}(\text{Sym}(\hat{\mathcal{D}}_j)) \|v\|_2^2$, hence $\hat{\mathcal{D}}_j$ is invertible with

$$\|\hat{\mathcal{D}}_j^{-1}\|_2 \leq \frac{1}{\lambda_{\min}(\text{Sym}(\hat{\mathcal{D}}_j))}. \quad (4.4)$$

Now, recall that $J^f = -D^{-1}(L+U)$, where D is the block-diagonal of H^f and $L+U$ is its off-diagonal part. Since H^f is block-tridiagonal, so is $L+U$, with $(L+U)_{j,j-1} = \hat{\mathcal{C}}_{j-1}^-$ and $(L+U)_{j,j+1} = \hat{\mathcal{C}}_j^+$. Left-multiplying by the block-diagonal matrix $-D^{-1}$ acts row-block-by-row-block: the j th row-block of $D^{-1}(L+U)$ picks up only the (j, j) -block of D^{-1} , namely $\hat{\mathcal{D}}_j^{-1}$, since every other column of row j of D^{-1} is zero. So the entire j th row of $L+U$ is just rescaled on the left by $\hat{\mathcal{D}}_j^{-1}$, and its sparsity pattern is preserved. Reading this off, J^f is block-tridiagonal with $(J^f)_{j,j-1} = -\hat{\mathcal{D}}_j^{-1} \hat{\mathcal{C}}_{j-1}^-$ and $(J^f)_{j,j+1} = -\hat{\mathcal{D}}_j^{-1} \hat{\mathcal{C}}_j^+$:

$$J^f = \begin{pmatrix} 0 & -\hat{\mathcal{D}}_1^{-1} \hat{\mathcal{C}}_1^+ & & \\ -\hat{\mathcal{D}}_2^{-1} \hat{\mathcal{C}}_1^- & 0 & -\hat{\mathcal{D}}_2^{-1} \hat{\mathcal{C}}_2^+ & \\ \ddots & \ddots & \ddots & \\ & & -\hat{\mathcal{D}}_{N-1}^{-1} \hat{\mathcal{C}}_{N-2}^- & 0 \end{pmatrix}$$

Since $\rho(J^f) \leq \|J^f\|$ in any induced operator norm, it suffices to bound $\|J^f\|$ in one convenient choice. The block-tridiagonal structure suggests the block max-norm $\|x\|_{\infty,2} = \max_j \|x_j\|_2$, where $x_j \in \mathbb{R}^n$ is the j th size- n block of x . Its induced operator form is

$$\|J^f\|_{\infty,2} = \sup_{x \neq 0} \frac{\max_j \|(J^f x)_j\|_2}{\|x\|_{\infty,2}}.$$

Therefore, bounding $\|(J^f x)_j\|_2$ by a multiple of $\|x\|_{\infty,2}$ bounds $\rho(J^f)$ by the same constant. Each block of $J^f x$ has form

$$(J^f x)_j = -\hat{\mathcal{D}}_j^{-1} \hat{\mathcal{C}}_{j-1}^- x_{j-1} - \hat{\mathcal{D}}_j^{-1} \hat{\mathcal{C}}_j^+ x_{j+1},$$

and submultiplicativity of the spectral norm together with (4.4) give

$$\begin{aligned} \|(J^f x)_j\|_2 &\leq \|\hat{\mathcal{D}}_j^{-1} \hat{\mathcal{C}}_{j-1}^-\|_2 \|x_{j-1}\|_2 + \|\hat{\mathcal{D}}_j^{-1} \hat{\mathcal{C}}_j^+\|_2 \|x_{j+1}\|_2 \\ &\leq \left(\|\hat{\mathcal{D}}_j^{-1} \hat{\mathcal{C}}_{j-1}^-\|_2 + \|\hat{\mathcal{D}}_j^{-1} \hat{\mathcal{C}}_j^+\|_2 \right) \|x\|_{\infty,2} \\ &\leq \left(\|\hat{\mathcal{D}}_j^{-1}\|_2 \|\hat{\mathcal{C}}_{j-1}^-\|_2 + \|\hat{\mathcal{D}}_j^{-1}\|_2 \|\hat{\mathcal{C}}_j^+\|_2 \right) \|x\|_{\infty,2} \\ &\leq \frac{\|\hat{\mathcal{C}}_{j-1}^-\|_2 + \|\hat{\mathcal{C}}_j^+\|_2}{\lambda_{\min}(\text{Sym}(\hat{\mathcal{D}}_j))} \|x\|_{\infty,2}. \end{aligned}$$

By (4.3), the fraction is strictly less than 1. Taking $\max_j$,

$$\rho(J^f) \leq \|J^f\|_{\infty,2} \leq \max_j \frac{\|\hat{C}_{j-1}^-\|_2 + \|\hat{C}_j^+\|_2}{\lambda_{\min}(\text{Sym}(\hat{\mathcal{D}}_j))} < 1.$$

Invertibility of every $\hat{\mathcal{D}}_j$ and $\rho(J^f) < 1$ are exactly the hypotheses of lemma 4, which gives local convergence of the forced Jacobi-Newton method to $\mathbf{q}^*$. $\square$

Remark 2 (Time complexity). Checking (4.3) reduces to a smallest-eigenvalue computation on $\text{Sym}(\hat{\mathcal{D}}_k)$ of runtime $O(n^3)$, and two spectral-norm computations on $\hat{C}_k^+$ and $\hat{C}_k^-$ ($O(n^3)$ each via SVD, cheaper via power iteration). Total cost is $O(Nn^3)$, of the same order as the unforced check and as the power-iteration estimate of $\rho(J^f)$ discussed at the start of the section.

The problem with this result is that it is very restrictive, in the sense that many convergent systems do not pass this criterion. This, together with the fact that it is not significantly cheaper than checking the spectral radius of J^f , makes this condition impractical to use. This result is provided as more of a theoretical result that matches theorem 2, and its use is in demonstrating ideas to bound the spectral radius, that hopefully can be built upon in the future.

4.2.2. A perturbation condition

In this section we obtain a condition on forces such that, if the unforced problem converges, then the forced system will too. The main idea is to decompose the Hessian as

$$H^f(\mathbf{q}) = H(\mathbf{q}) + \Delta F(\mathbf{q}), \quad (4.5)$$

where H is the unforced Hessian (3.8) and ΔF is a block-tridiagonal *force-perturbation matrix*, with blocks

$$\Delta \hat{\mathcal{D}}_k = D_2 f_d^+(q_{k-1}, q_k) + D_1 f_d^-(q_k, q_{k+1}),$$

$$\Delta \hat{C}_k^+ = D_2 f_d^-(q_k, q_{k+1}),$$

$$\Delta \hat{C}_k^- = D_1 f_d^+(q_k, q_{k+1}).$$

Notice that $\Delta F = 0$ when all forces vanish, recovering $H^f = H$, so ΔF is a measure of the deviation from the unforced problem.

The following theorem states in a rigorous and quantitative manner the intuitive idea that, if the algorithm already converges without forces, then it still converges when forces are added, as long as they are “weak enough”, which manifest as the derivatives $D_i f_d^\pm$ that appear in the Jacobian being small “enough”.

Theorem 6 (Perturbation condition). *Suppose the unforced Jacobi-Newton method converges at $\mathbf{q}^*$: the diagonal D of $H(\mathbf{q}^*)$ is invertible and $\rho(J(\mathbf{q}^*)) < 1$. Then there exists an $\varepsilon > 0$ such that, whenever $\|\Delta F(\mathbf{q}^*)\|_2 < \varepsilon$, the forced Jacobi-Newton method also converges locally to $\mathbf{q}^*$.*

Proof. Let D, L, U denote the block-diagonal, strict block-lower, and strict block-upper parts of the unforced Hessian H , and similarly $\Delta D, \Delta L, \Delta U$ for the perturbation ΔF . Then the forced iteration matrix is

$$J^f = -(D + \Delta D)^{-1} ((L + U) + (\Delta L + \Delta U)),$$

and the unforced one ($\Delta F = 0$) is $J = -D^{-1}(L + U)$ with $\rho(J) < 1$ by hypothesis.

First, notice that the map $\Delta F \mapsto J^f$ is continuous at $\Delta F = 0$. This is because each D_k is invertible by hypothesis, so $\|D^{-1}\|_2$ is finite block-wise. Since ΔD is the block-diagonal part of ΔF , the spectral norm satisfies $\|\Delta D\|_2 \leq \|\Delta F\|_2$ (block-diagonal projection does not increase the spectral norm). Hence for $\|\Delta F\|_2 < \|D^{-1}\|_2^{-1}$, we have that $\|\Delta D\|_2 < \|D^{-1}\|_2^{-1}$. Then, we can write the Neumann series ([10, Corollary 5.6.16])

$$(D + \Delta D)^{-1} = D^{-1} \sum_{n \geq 0} (-\Delta D D^{-1})^n$$

which converges uniformly on a neighborhood of $\Delta F = 0$. The series itself is the inverse, establishing that $D + \Delta D$ is invertible there; and since each partial sum is a polynomial (hence, continuous) function of ΔD , uniform convergence implies that $(D + \Delta D)^{-1}$ depends continuously on ΔF . Matrix multiplication and addition are continuous, so J^f also depends continuously on ΔF on the same neighborhood. In particular, $J^f \rightarrow J$ in any matrix norm as $\|\Delta F\| \rightarrow 0$.

Notice also that the spectral radius ρ cannot jump upward under small perturbations of its matrix argument. By Gelfand's formula [10, Cor. 5.6.14],

$$\rho(A) = \inf_{k \geq 1} \|A^k\|^{1/k}.$$

Each $A \mapsto \|A^k\|^{1/k}$ is continuous in A , and an infimum of continuous functions is upper semicontinuous. Concretely, for any matrix A_0 and any $\eta > \rho(A_0)$, there is a neighborhood of A_0 (in $M_n(\mathbb{R})$) where $\rho(A) < \eta$.

Combining these two observations, since $\rho(J) < 1$, there is some η with $\rho(J) < \eta < 1$. Upper semicontinuity applied at $A_0 = J$ gives a neighborhood $U \subset M_n(\mathbb{R})$ of J on which $\rho(A) < \eta$. By continuity of $\Delta F \mapsto J^f$, the preimage $V' = (J^f)^{-1}(U)$ is open in perturbation-space and contains $\Delta F = 0$. Invertibility of $D + \Delta D$ holds on the open set $V'' = \{\Delta F : \|\Delta F\|_2 < \|D^{-1}\|_2^{-1}\}$, also a neighborhood of $\Delta F = 0$. Their intersection $V = V' \cap V''$ is therefore an open neighborhood of $\Delta F = 0$ on which both $J^f \in U$ and $D + \Delta D$ is invertible. Since V is open and contains 0, it contains an open ball of some radius $\varepsilon > 0$ around 0, so $\|\Delta F\|_2 < \varepsilon$ implies

$$\rho(J^f) < \eta < 1 \quad \text{and} \quad D + \Delta D \text{ is invertible.}$$

These are exactly the hypotheses of lemma 4, which then gives local convergence of the forced Jacobi-Newton method to $\mathbf{q}^*$.

□

In theorem 6 we only prove that such an ε exists. In the following lemma, we find an explicit computation, under the mild additional assumption that $\|J\| < 1$ holds in some submultiplicative matrix norm (i.e., a norm such that $\|AB\| \leq \|A\|\|B\|$).

Lemma 7 (Explicit perturbation bound). *Assume the hypotheses of theorem 6, and further that $\|J\| < 1$ in a submultiplicative norm $\|\cdot\|$ in which the block-diagonal projection is contractive (e.g. the spectral norm). Then the conclusion of the theorem holds with*

$$\varepsilon = \frac{1 - \|J\|}{3 \|D^{-1}\|}. \quad (4.6)$$

Proof. Let $\beta = \|D^{-1}\|$ and $\gamma = \|\Delta F\|$. The identity $A^{-1} - B^{-1} = A^{-1}(B - A)B^{-1}$, combined with $J = -D^{-1}(L + U)$, gives by direct manipulation

$$\begin{aligned} J^f - J &= \underbrace{-(D + \Delta D)^{-1} [(L + U) + (\Delta L + \Delta U)]}_{J^f} + \underbrace{D^{-1}(L + U)}_{-J} \\ &= -[(D + \Delta D)^{-1} - D^{-1}](L + U) - (D + \Delta D)^{-1}(\Delta L + \Delta U) \\ &= (D + \Delta D)^{-1} \Delta D D^{-1}(L + U) - (D + \Delta D)^{-1}(\Delta L + \Delta U) \\ &= -(D + \Delta D)^{-1} \Delta D \cdot J - (D + \Delta D)^{-1}(\Delta L + \Delta U) \\ &= -(D + \Delta D)^{-1} [\Delta D \cdot J + (\Delta L + \Delta U)]. \end{aligned}$$

The three factors on the right are bounded by, respectively, $\|\Delta D\| \leq \gamma$ (projection contractivity), $\|\Delta L + \Delta U\| = \|\Delta F - \Delta D\| \leq 2\gamma$ (triangle inequality), and $\|(D + \Delta D)^{-1}\| \leq \beta/(1 - \beta\gamma)$ (Neumann tail estimate², valid for $\beta\gamma < 1$). Combining,

$$\|J^f - J\| \leq \frac{\beta\gamma(\|J\| + 2)}{1 - \beta\gamma}.$$

Convergence follows from $\|J^f\| < 1$, which holds if $\|J^f - J\| \leq 1 - \|J\|$. Imposing this on the bound above and solving for γ :

$$\begin{aligned} \frac{\beta\gamma(\|J\| + 2)}{1 - \beta\gamma} &\leq 1 - \|J\| \\ \beta\gamma(\|J\| + 2) &\leq (1 - \|J\|)(1 - \beta\gamma) \\ \beta\gamma(\|J\| + 2 + (1 - \|J\|)) &\leq 1 - \|J\| \\ 3\beta\gamma &\leq 1 - \|J\| \\ \gamma &\leq \frac{1 - \|J\|}{3\beta}, \end{aligned}$$

results in (4.6). □

²We factor $D + \Delta D = D(I + D^{-1}\Delta D)$, so $(D + \Delta D)^{-1} = (I + D^{-1}\Delta D)^{-1}D^{-1}$. Since $\|D^{-1}\Delta D\| \leq \beta\gamma < 1$, the Neumann series $(I + M)^{-1} = \sum_{k \geq 0} (-M)^k$ converges and $\|(I + D^{-1}\Delta D)^{-1}\| \leq \sum_{k \geq 0} (\beta\gamma)^k = 1/(1 - \beta\gamma)$. Multiplying by $\|D^{-1}\| = \beta$ gives the claimed bound.

We call the factor $1 - \|J\|$ in (4.6) the *contraction margin* of the unforced iteration, and $\|D^{-1}\|$ measures how well-conditioned the diagonal is. A smaller margin or a worse-conditioned diagonal will shrink ε , making the margin of the magnitude of allowed forces tighter. This perturbation bound is checkable in terms of derivatives of the discrete forces $f_d^\pm$ alone, which depend on the discrete Lagrangian and the chosen discretization of the force, but not on the (unknown) solution trajectory (although it does not say anything about whether the unforced system converges in the first place).

In practice, however, this guarantee is pessimistic, and (somewhat counterintuitively) it weakens as the discretization is refined. For many problems, as $h \rightarrow 0$ the contraction margin $1 - \|J\|$ decreases faster than $\|D^{-1}\|$, so $\varepsilon \rightarrow 0$ even though the true admissible force grows. This is shown in example 4.3.1.

Remark 3 (Time complexity). Checking the bound (4.6) requires, in terms of the path length N and the configuration space dimension n :

- $\|D^{-1}\|_2 = \max_k \|\hat{D}_k^{-1}\|_2$: N independent $n \times n$ block inversions, costing $O(Nn^3)$.
- $\|J\|_2$ via power iteration on $J^T J$: $O(kNn^2)$ for k iterations.
- $\|\Delta F\|_2$ at the chosen point: same cost as $\|J\|_2$, $O(kNn^2)$.

The dominant cost is therefore $O(kNn^2)$ (since usually $k \gtrsim n$). I.e., linear in path size, which makes it practical in terms of computation.

4.3. Examples

We illustrate the forced algorithm and the convergence criteria on two systems: example 4.3.1 applies the perturbation theorem to a simple dissipative force, and example 4.3.2 shows a practical application of the algorithm to find trajectories for a parachutist in turbulent wind conditions.

The source code, including all the code and parameter setup for the following examples, is available in the repository at <https://github.com/sebajf/pdel-mechanics> (GitHub) and <https://git-archive.odilf.com/odilf/pdel-mechanics> (archived snapshot).

4.3.1. Damped harmonic oscillator

Consider a one-dimensional harmonic oscillator with linear viscous damping,

$$\ddot{q} + \gamma \dot{q} + \omega^2 q = 0, \quad \gamma > 0,$$

which has the following Lagrangian:

$$L(q, \dot{q}) = \frac{1}{2}\dot{q}^2 - \frac{1}{2}\omega^2 q^2$$

$$f^L(q, \dot{q}) = -\gamma\dot{q}.$$

We discretize the Lagrangian and the force with the trapezoidal rule, evaluating both at the two endpoints q_k, q_{k+1} (since the damping is velocity-only, the force quadrature here coincides with the trapezoidal quadrature):

$$L_d(q_k, q_{k+1}) = \frac{(q_{k+1} - q_k)^2}{2h} - \frac{\omega^2 h}{4}(q_k^2 + q_{k+1}^2)$$

$$f_d^+(q_k, q_{k+1}) = -\frac{\gamma}{2}(q_{k+1} - q_k)$$

$$f_d^-(q_k, q_{k+1}) = -\frac{\gamma}{2}(q_{k+1} - q_k).$$

The ‘‘blocks’’ of the forced Hessian are just scalars, which by (4.2) evaluate to

$$\hat{\mathcal{A}}_k = \frac{1}{h} - \frac{\omega^2 h}{2} + \frac{\gamma}{2} \quad \hat{\mathcal{C}}_k^+ = -\frac{1}{h} - \frac{\gamma}{2}$$

$$\hat{\mathcal{B}}_k = \frac{1}{h} - \frac{\omega^2 h}{2} - \frac{\gamma}{2} \quad \hat{\mathcal{C}}_k^- = -\frac{1}{h} + \frac{\gamma}{2}.$$

Comparing with the unforced blocks $\mathcal{A}_k = \mathcal{B}_k = \frac{1}{h} - \frac{\omega^2 h}{2}$ and $\mathcal{C}_k^\pm = -\frac{1}{h}$, the force perturbation matrix ΔF has

$$\Delta \hat{\mathcal{D}}_k = D_2 f_d^+(q_{k-1}, q_k) + D_1 f_d^-(q_k, q_{k+1}) = -\frac{\gamma}{2} + \frac{\gamma}{2} = 0.$$

That is, the trapezoidal quadrature leaves the diagonal blocks of H^f unchanged relative to the unforced system and the damping force enters H^f only through the off-diagonal blocks,

$$\Delta \hat{\mathcal{C}}_k^+ = -\frac{\gamma}{2}, \quad \Delta \hat{\mathcal{C}}_k^- = \frac{\gamma}{2}.$$

Thus ΔF is purely off-diagonal with antisymmetric blocks ($\Delta \hat{\mathcal{C}}_k^- = -\Delta \hat{\mathcal{C}}_k^+$), so the skew part of H^f grows with γ while the symmetric part remains that of the unforced oscillator. Here, $\|\Delta F\|_2$ can be computed exactly. $\Delta F = \frac{\gamma}{2}S$, where S is the $(N-1) \times (N-1)$ skew-symmetric tridiagonal Toeplitz matrix with -1 on the super-diagonal and $+1$ on the sub-diagonal, which has known eigenvalues $\lambda_j = 2i \cos \frac{j\pi}{N}$ for $j = 1, \dots, N-1$. Since S is normal (being skew-symmetric), its singular values are $|\lambda_j|$, so $\|S\|_2 = \max_j |\lambda_j| = 2 \cos \frac{\pi}{N}$ and

$$\|\Delta F\|_2 = \gamma \cos \frac{\pi}{N}.$$

Assuming the unforced oscillator converges at $\mathbf{q}^*$ for the chosen step size h , theorem 6 and lemma 7 guarantee that the forced method also converges whenever

$$\gamma \cos \frac{\pi}{N} < \varepsilon = \frac{1 - \|J^0\|}{3\|D_0^{-1}\|} \implies \gamma < \frac{\varepsilon}{\cos(\pi/N)},$$

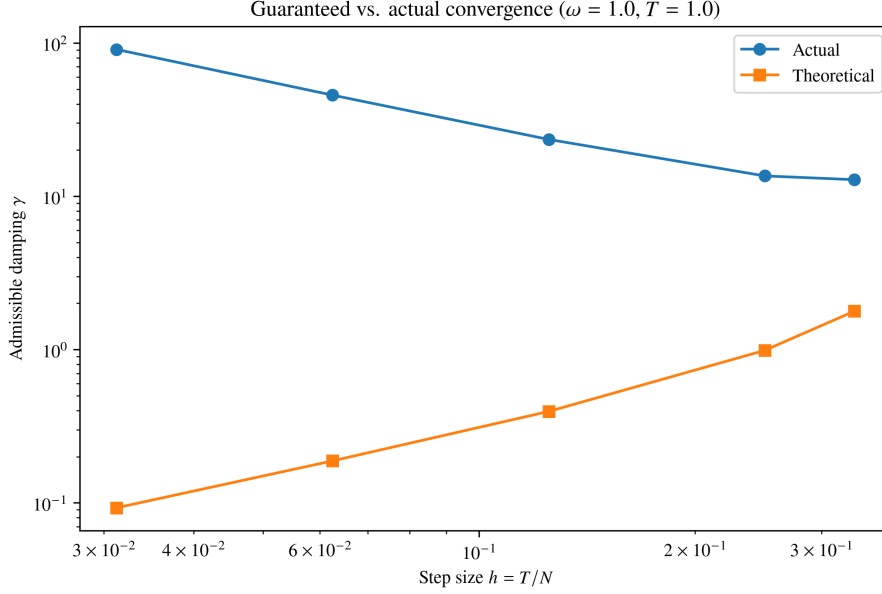


Fig. 4.1. Guaranteed vs actual convergence threshold for a damped oscillator depending on step size $h = T/N$. The bound $\varepsilon/\cos(\pi/N)$ of lemma 7 is plotted against the true threshold (the largest γ with $\rho(J^f) < 1$, located by bisection). The guarantee is not only conservative, but in fact moves the wrong way under refinement: as $h \rightarrow 0$ the admissible γ decreases, while the true threshold increases.

where J^0 and D_0^{-1} are the Jacobi matrix and inverse diagonal of the unforced system at $\mathbf{q}^*$. This gives a concrete bound on the admissible damping coefficient in terms of the step size and the unforced convergence margin.

Figure 4.1 compares this bound against the true convergence threshold as the step size is refined. We see that the two diverge as $h \rightarrow 0$, with the guaranteed γ decreasing at order h while the actual threshold grows at order $1/h$ (so the true limit would only be reached if γh was of order one).

Fundamentally, this discrepancy is due to the lemma bounding $\|J^f\| \leq \|J\| + \|J^f - J\|$ and treating $\hat{C}^+$ and $\hat{C}^-$ independently: $\Delta\hat{C}^+ = -\gamma/2$ and $\Delta\hat{C}^- = +\gamma/2$, so each contributes $\gamma/2$ of norm to $\|J^f - J\|$, giving a total first-order cost $\|J^f - J\| \sim \gamma h$. For this oscillator the unforced iteration has $\|J\| = \cos(\pi/N)$ (the $\gamma \rightarrow 0$ limit of $\rho(J^f)$ below), so the contraction margin is $1 - \|J\| = 1 - \cos(\pi/N) \approx \frac{\pi^2}{2N^2} = \frac{\pi^2 h^2}{2T^2} = O(h^2)$. With this $O(h^2)$ margin, $\|J^f - J\| \leq 1 - \|J\|$ forces $\gamma h \lesssim h^2$, i.e. $\gamma \lesssim h$. But, in reality, the two contributions cancel at first order. The eigenvalues of J^f depend on the *product* of the off-diagonal blocks:

$$\sqrt{\hat{C}^+ \hat{C}^-} = \sqrt{\left(\frac{1}{h} + \frac{\gamma}{2}\right)\left(\frac{1}{h} - \frac{\gamma}{2}\right)} = \frac{1}{h} \sqrt{1 - \frac{\gamma^2 h^2}{4}},$$

so the $\pm\gamma/2$ shifts cancel and γ enters only at second order. The spectral radius is then $\rho(J^f) \approx \cos(\pi/N) \sqrt{1 - \gamma^2 h^2/4}$, which only crosses 1 near $\gamma h \approx 2\sqrt{2}$, giving the true threshold $\gamma \sim 1/h$, and hence the $\sim N^2$ divergence in fig. 4.1.

4.3.2. Parachutist in turbulent wind

A parachutist wants to jump from a helicopter at some position (x_0, y_0) and reach a target point on the ground $(x_T, 0)$ at some time T by letting themselves be carried by the wind, which is given as some vector field $W(x, y) = (w_x, w_y)$.

This problem can be modeled with the Lagrangian

$$L(x, y, \dot{x}, \dot{y}) = \frac{m}{2}(\dot{x}^2 + \dot{y}^2) - mgy$$

where g is the gravitational acceleration and m is the diver's mass. Furthermore, the parachutist feels some drag, which is modeled as an external force:

$$(F_x, F_y) = (-k(\dot{x} - w_x(x, y)), -k(\dot{y} - w_y(x, y))),$$

where k is the parachute drag coefficient.

The problem can be discretized with the midpoint rule, for instance. The only additional data needed are the length of the path N , which then gives us the timestep $h = T/N$. Finally, solving it allows us to recover what the initial velocity must be to reach the desired endpoint by just being dragged by the wind.

The results of the implementation are shown in figs. 4.2 and 4.3. Figure 4.2 shows how the trajectory starts as a straight line to the endpoint, but it curves upwards to be made physical. It turns out that for this particular wind field that has two vortices, the parachutist needs to jump with velocity mostly to the right, and slightly upwards.

Figure 4.3 shows many trajectories from different starting positions, both for the vortex wind field (fig. 4.3a) as well as for a wind field that forms a shear (fig. 4.3b). This serves to illustrate a couple non-trivial observations. For instance, trajectory 3 (hexagon) in fig. 4.3a needs a very high initial velocity, which indicates that it is most likely not realistic for the parachutist to jump from that point and reach the target; and trajectory 2 (star) in fig. 4.3b shows that to reach the endpoint the parachutist has to first jump *away* from the target to reach it in the desired time (i.e., jump up and to the right, while the target is down and to the left).

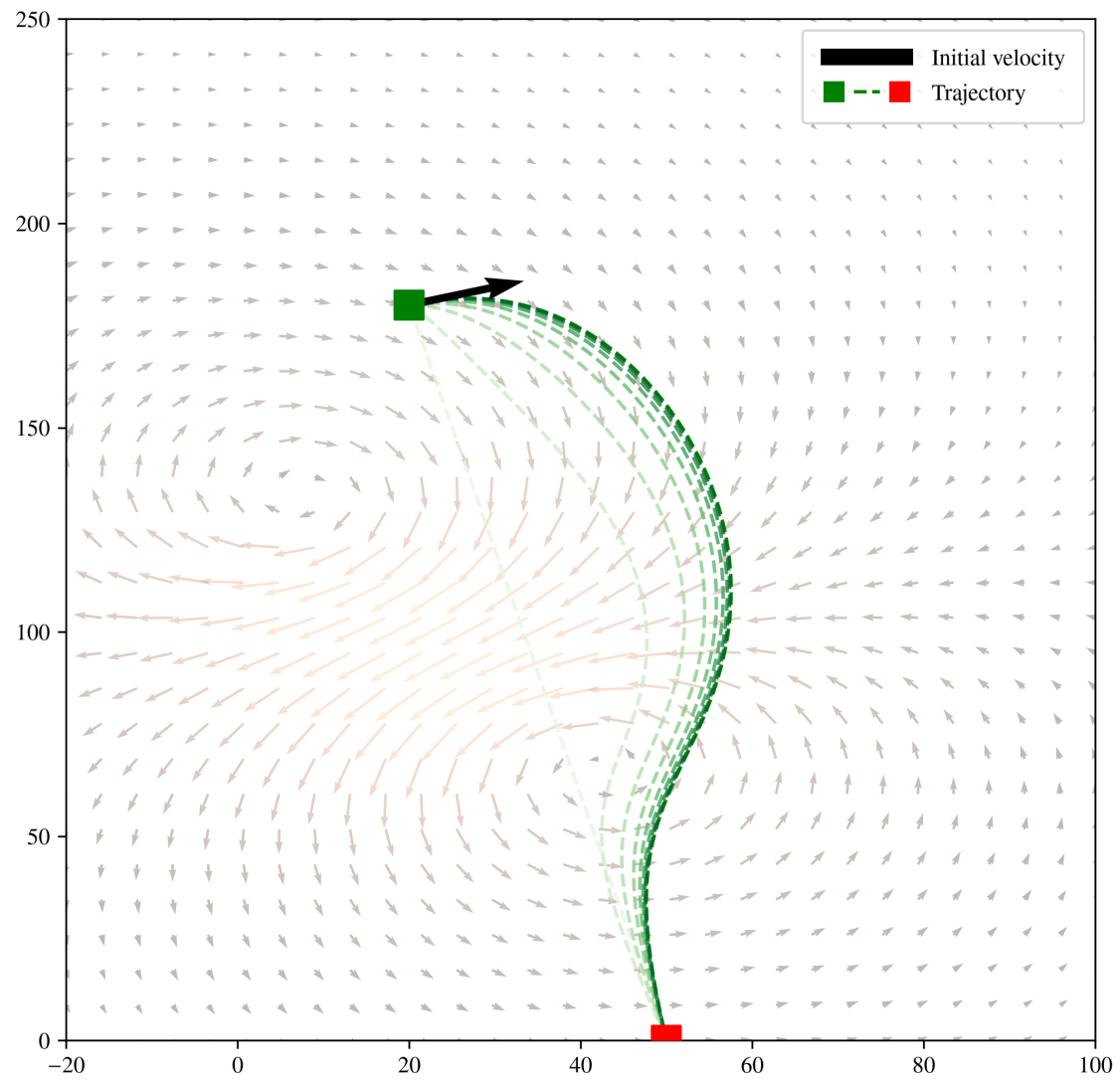
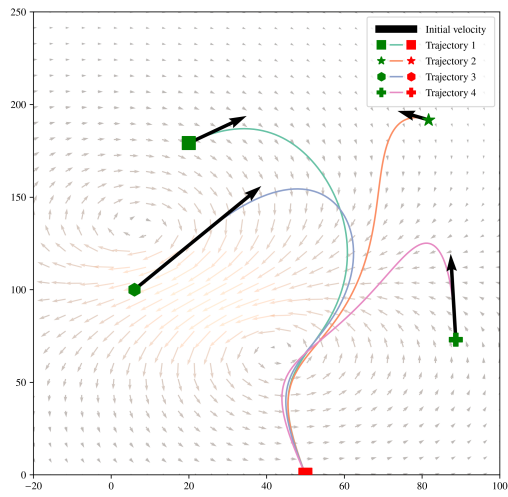
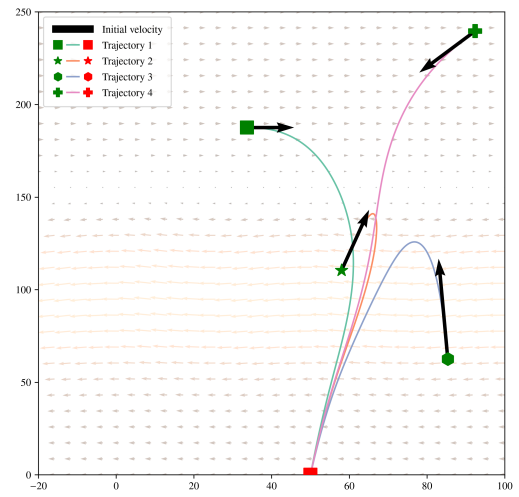


Fig. 4.2. Trajectory and initial velocity that a parachutist needs to reach a desired endpoint in a wind field with two vortices. Green path indicates trajectory, fainter green paths indicate intermediate solver iterations.



(a) Vortex wind field (same as fig. 4.2)



(b) Shear wind field

Fig. 4.3. Trajectories and initial velocities needed to reach an endpoint, depending on starting position.

5. LIE GROUPS

Many physical systems are naturally defined on a configuration manifold where no notion of linear combination can exist, thus not being able to be described as a vector space. However, most of these still have some structure; namely, that of a *Lie group* G : a smooth manifold together with a group operation $\cdot : G \times G \rightarrow G$ such that multiplication and inversion are smooth. Examples of such manifolds are the rotation groups $SO(n)$, the Euclidean motion groups $SE(n)$, the unitary groups $U(n)$, and in fact every vector space (under addition).

In this chapter we extend the parallel algorithm (section 3.4) to work on Lie groups.

5.1. The parallel algorithm on Lie algebras

Where the vector space step fails The Jacobi step of the parallel algorithm, written for an interior node k and a current path $\mathbf{g}$, looks the same as before:

$$r_k(\mathbf{g}, \bar{g}_k) = D_2 L_d(g_{k-1}, \bar{g}_k) + D_1 L_d(\bar{g}_k, g_{k+1}) = 0.$$

In this setting, taking the derivative with $g_{k\pm 1}$ fixed entails going from the two partially-applied Lagrangians $L_d^-(\cdot) = L_d(g_{k-1}, \cdot)$ and $L_d^+(\cdot) = L_d(\cdot, g_{k+1})$, each a map $G \rightarrow \mathbb{R}$, to $T_{\bar{g}_k} G \rightarrow T_{L_d^{\pm}(\bar{g}_k)} \mathbb{R}$, and using the fact that every tangent space of $\mathbb{R}$ is canonically isomorphic to $\mathbb{R}$, this becomes a sum in $T_{\bar{g}_k}^* G$. This forms a vector space so this step is well-defined.

What does fail is the Newton update (3.7):

$$\bar{q} = q_k - (D_2 r_k(\mathbf{q}, q_k))^{-1} r_k(\mathbf{q}, q_k),$$

because this uses subtraction in a space G where no canonical subtraction is available.

One solution to this is working on the corresponding *Lie algebra*. The Lie algebra $\mathfrak{g} = T_e G$ is the tangent space at the identity, forming a vector space of the same dimension as G . For any other point $g \in G$, tangent directions can be supplied by the differential $T_e L_g : \mathfrak{g} \rightarrow T_g G$ of the left-translation map $L_g : h \mapsto gh$, which is an isomorphism between these two vector spaces. The technique of “moving” some computation of a Lie group to its algebra is often called *lifting* (into the algebra).

In the vector space case, the Newton update has form of $q_{\text{new}} = q_k + \delta q$: you compute a correction δq in a vector space and add it. On G , there is no addition, so “compute a correction and add it” is not well-defined. Instead, we pick a vector space (here, $\mathfrak{g}$), compute a correction $\delta \xi$ there, and then map it to a nearby point in G via some smooth map.

Such a map is called a *retraction* $\tau : \mathfrak{g} \rightarrow G$, which (for our purposes) we require to satisfy $\tau(0) = e$ and $T_0\tau = \text{Id}_{\mathfrak{g}}$. Left-translating τ by g_k gives a map centred at g_k :

$$\bar{g}_k = g_k \tau(\delta\xi_k), \quad \delta\xi_k \in \mathfrak{g}, \quad (5.1)$$

the Lie group analog of $q_k + \delta q$ in the vector space case. The correction $\delta\xi_k$ lives in the vector space $\mathfrak{g}$ and can be solved for by a linear system (which we show how to set up in the next section).

Remark 4 (Concrete retractions). The *exponential map* $\exp : \mathfrak{g} \rightarrow G$ is the fundamental and universal retraction; but, for instance, in matrix Lie groups the *Cayley map* $\text{cay}(\xi) = (I - \xi/2)^{-1}(I + \xi/2)$ is often used instead, since it has some nicer properties, such as being easier to compute.

Linearization As it stands, the residual $r_k(\mathbf{g}, g_k \tau(\delta\xi))$ is an element of $T_{g_k \tau(\delta\xi)}^* G$. We are going to linearize it in the Lie algebra by considering the expression $r_k(\mathbf{g}, g_k \tau(\delta\xi))$ as a function of a vector direction $\delta\xi \in \mathfrak{g}$. This gives a map $\mathfrak{g} \rightarrow T_{g_k \tau(\delta\xi)}^* G$, i.e., from the algebra to a cotangent space. Similarly to how we can move from $T_g G \rightarrow \mathfrak{g}$ via $T_e L_g$, we can move from $T_g^* G \rightarrow \mathfrak{g}^*$ using the *transpose* $(T_e L_g)^*$ by interpreting the duals as linear form and pre-applying $T_e L_g$ to $T_g^* G$. That is,

$$((T_e L_g)^* \underbrace{\alpha}_{T_g^* G}) \underbrace{\eta}_{T_g G} = \alpha(T_e L_g(\eta)).$$

(the transpose is a specific case of a pullback, see 2.1.9).

Note that since L_g is a diffeomorphism, both $T_e L_g$ and its transpose are isomorphisms and so this mapping can be done without loss of information.

Then, the residual can be written as a function of a vector direction, all in the Lie algebra:

$$\begin{aligned} \tilde{r}_k : \mathfrak{g} &\rightarrow \mathfrak{g}^* \\ \tilde{r}_k(\delta\xi_k) &= (T_e L_{g_k \tau(\delta\xi_k)})^* r_k(\mathbf{g}, g_k \tau(\delta\xi_k)), \end{aligned} \quad (5.2)$$

This expression is linearizable around $\delta\xi_k = 0$ as $\partial_{\delta\xi_k} \tilde{r}_k|_0 \delta\xi_k + \tilde{r}_k(0)$. To carry out the computations we will follow a common convention of denoting $T_e L_g(\eta) = g \cdot \eta$, which in the general case is no more than notational shorthand, but for matrix Lie groups the pushforwarded translation does literally correspond to matrix multiplication, denoted with the same symbol.

The residual $r_k(\mathbf{g}, g_k \tau(\delta\xi))$ takes values in the cotangent fibre $T_{g_k \tau(\delta\xi)}^* G$, which moves with $\delta\xi$; elements of distinct fibres cannot be compared canonically, so $\partial_{\delta\xi} r_k$ is not by itself well-defined. What we differentiate instead is the trivialized residual $\tilde{r}_k(\delta\xi) \in \mathfrak{g}^*$ of (5.2), which does live in a fixed vector space. For a direction $\eta \in \mathfrak{g}$, the result is

$$\partial_{\delta\xi} \tilde{r}_k|_0 [\eta] = \underbrace{(T_e L_{g_k})^* \frac{d}{dt} r_k(\mathbf{g}, g_k \tau(t\eta)) \Big|_{t=0}}_{\text{(I)}} + \underbrace{\left(\frac{d}{dt} (T_e L_{g_k \tau(t\eta)})^* \Big|_{t=0} \right) r_k(\mathbf{g}, g_k)}_{\text{(II)}}$$

where (I) differentiates r_k in the frame at g_k and (II) differentiates the moving frame itself. Term (II) is proportional to the untrivialized residual $r_k(\mathbf{g}, g_k)$, which vanishes at every solution $\mathbf{g}^*$ of the DEL equations. We keep only term (I), making the iteration an approximate Newton method; since (II) vanishes at solutions, the operator we invert agrees with the exact linearization there, and so the fixed points of the iteration coincide with solutions of the DEL equations.

Term (I) evaluates via the chain rule to

$$\left. \frac{d}{dt} r_k(\mathbf{g}, g_k \tau(t\eta)) \right|_{t=0} = (D_{22}L_d(g_{k-1}, g_k) + D_{11}L_d(g_k, g_{k+1})) [g_k \cdot \eta].$$

Pulling this back with $(T_e L_{g_k})^*$ and setting the result to zero gives the linear system

$$\tilde{\mathcal{D}}_k \delta \xi_k = -\tilde{r}_k, \quad (\text{Jacobi-Newton-G})$$

where $\tilde{\mathcal{D}}_k : \mathfrak{g} \rightarrow \mathfrak{g}^*$ at node k is

$$\tilde{\mathcal{D}}_k = (T_e L_{g_k})^* (D_{22}L_d(g_{k-1}, g_k) + D_{11}L_d(g_k, g_{k+1})) (T_e L_{g_k}) \quad (5.3)$$

(it will be shown later that this is in fact the diagonal blocks of the Lie algebra Hessian at $\mathbf{g}^*$, as in the vector space case; the discrepancy between $\tilde{\mathcal{D}}_k$ and the true Hessian diagonal away from $\mathbf{g}^*$ is the retraction-curvature term, which vanishes at solutions).

Finally, the update rule is

$$\bar{g}_k = g_k \tau \left(-\tilde{\mathcal{D}}_k^{-1} \tilde{r}_k \right). \quad (5.4)$$

The global parallel loop is unchanged from the vector space and forced cases: each inner node k independently solves (**Jacobi-Newton-G**), and the results are assembled into the next iterate path $\bar{\mathbf{g}}$. The only difference is that the local solve happens in $\mathfrak{g}$, and the update uses the retraction τ instead of vector addition. The parallelism structure (section 3.4) carries over without modification. The treatment of forces (chapter 4) can also be applied in the same way, namely by inserting the discrete forces $f_d^\pm$ into the residual r_k before taking the Lie algebra pullback.

5.2. Convergence

The convergence analysis largely follows the vector space case (theorem 1) from [1], but with two adaptations: we work with the Lie algebra Hessian $\tilde{H}$ (the analog of H , with all blocks pulled back to $\mathfrak{g}$ via left translation), and the iteration on G^{N-1} has to be read in a Lie algebra chart at the solution before we can apply classical fixed-point/convergence theorems.

The strategy consists of lifting the discrete action to a function on the vector space $\mathfrak{g}^{N-1}$, computing its gradient and Hessian there, and reading off the convergence criterion. We carry out the lifting in section 5.2.1 and give the convergence results in section 5.2.2.

5.2.1. The Hessian in Lie algebra coordinates

The Hessian cannot be computed on the Lie group directly (neither the gradient, for that matter), but it can be written in the Lie algebra. To do this, we fix the current iterate $\mathbf{g} = (g_0, g_1, \dots, g_N)$ with endpoints g_0 and g_N pinned, and for each interior node $k = 1, \dots, N-1$, we introduce a perturbation $\xi_k \in \mathfrak{g}$ and define the perturbed path

$$\mathbf{g}(\xi) = (g_0, g_1\tau(\xi_1), \dots, g_{N-1}\tau(\xi_{N-1}), g_N), \quad \xi \in \mathfrak{g}^{N-1},$$

with $\xi_0 = \xi_N = 0$ at the (fixed) endpoints. The discrete action restricted to this family is now a smooth function on the *vector space* $\mathfrak{g}^{N-1}$:

$$\Sigma(\xi) = S_d(\mathbf{g}(\xi)) = \sum_{k=0}^{N-1} L_d(g_k\tau(\xi_k), g_{k+1}\tau(\xi_{k+1})). \quad (5.5)$$

This puts us back in the vector space setting of section 3.4, where we will show that the gradient and Hessian of Σ at $\xi = 0$ play the same role that ∇S_d and $\nabla^2 S_d$ played there. We write

$$\tilde{R} = \nabla \Sigma(0) \in (\mathfrak{g}^*)^{N-1}, \quad \tilde{H} = \nabla^2 \Sigma(0).$$

Differentiating (5.5) with respect to ξ_k , only the $(k-1)$ -th and k -th summands contribute. Both apply L_d through the chart $\xi_k \mapsto g_k\tau(\xi_k)$, so the chain rule gives, at any ξ ,

$$\begin{aligned} \partial_{\xi_k} \Sigma(\xi) &= \Phi_k(\xi_k)^* r_k(\mathbf{g}(\xi), g_k\tau(\xi_k)) \\ \Phi_k(\xi_k) &= T_{\tau(\xi_k)} L_{g_k} T_{\xi_k} \tau, \end{aligned} \quad (5.6)$$

where $\Phi_k(\xi_k) : \mathfrak{g} \rightarrow T_{g_k\tau(\xi_k)} G$ is the differential of the chart and $r_k(\mathbf{g}, g_k)$ is the untrivialized residual. At $\xi = 0$, $\Phi_k(0)$ simplifies to $T_e L_{g_k}$, recovering the Lie algebra residual (5.2):

$$\partial_{\xi_k} \Sigma(0) = (T_e L_{g_k})^* (D_2 L_d(g_{k-1}, g_k) + D_1 L_d(g_k, g_{k+1})) = \tilde{r}_k.$$

In particular, solutions of the DEL equations on G are exactly the critical points of Σ .

The Hessian computation needs the second derivative of $g\tau(\xi)$ at $\xi = 0$, which is a bilinear form on tangent vectors:

$$D^2(g\tau)(0) = g \cdot D^2\tau(0). \quad (5.7)$$

The actual value of this term depends on the retraction. For the exponential map in matrix Lie groups, $D^2 \exp(0)[\eta, \zeta] = \frac{1}{2}(\eta\zeta + \zeta\eta)$. The Cayley map also evaluates to this, since it happens to agree with $\exp$ up to second order.

The diagonal block $\tilde{D}_k = \partial_{\xi_k}^2 \Sigma(0)$ picks up contributions from the two summands containing ξ_k (just as in eq. (3.8)). For each, the chain rule gives a ‘‘Hessian of L_d ’’ term plus a ‘‘retraction-curvature’’ term coming from (5.7). That is, for the $(k-1)$ -th summand:

$$\begin{aligned} \partial_{\xi_k}^2 L_d(g_{k-1}, g_k\tau(\xi_k)) \Big|_0 [\eta, \zeta] &= D_{22} L_d(g_{k-1}, g_k) [T_e L_{g_k} \eta, T_e L_{g_k} \zeta] \\ &\quad + D_2 L_d(g_{k-1}, g_k) [g_k \cdot D^2\tau(0)[\eta, \zeta]], \end{aligned}$$

and similarly for the k -th summand:

$$\begin{aligned} \partial_{\xi_k}^2 L_d(g_k \tau(\xi_k), g_{k+1}) \Big|_0 [\eta, \zeta] &= D_{11} L_d(g_k, g_{k+1}) [T_e L_{g_k} \eta, T_e L_{g_k} \zeta] \\ &\quad + D_{12} L_d(g_k, g_{k+1}) [g_k \cdot D^2 \tau(0) [\eta, \zeta]], \end{aligned}$$

Summing the two retraction-curvature terms collects $D_2 L_d(g_{k-1}, g_k) + D_1 L_d(g_k, g_{k+1}) = r_k(\mathbf{g}, g_k)$, so at a solution $\mathbf{g}^*$ where $r_k(\mathbf{g}^*, g_k^*) = 0$ the curvature term vanishes and

$$\tilde{\mathcal{D}}_k \Big|_{\mathbf{g}^*} = (T_e L_{g_k^*})^* (D_{22} L_d(g_{k-1}^*, g_k^*) + D_{11} L_d(g_k^*, g_{k+1}^*)) (T_e L_{g_k^*}),$$

which matches (5.3) from section 5.1, confirming that the Hessian of the lifted action agrees with the Newton step at the solution.

The off-diagonal block $\tilde{\mathcal{C}}_k = \partial_{\xi_k} \partial_{\xi_{k+1}} \Sigma(0)$ picks up only one summand (the k -th), and since the parametrisation $(\xi_k, \xi_{k+1}) \mapsto (g_k \tau(\xi_k), g_{k+1} \tau(\xi_{k+1}))$ is a product of independent retractions, there is no retraction-curvature term:

$$\tilde{\mathcal{C}}_k = (T_e L_{g_k})^* D_{12} L_d(g_k, g_{k+1}) (T_e L_{g_{k+1}}). \quad (5.8)$$

This expression holds at every path, not just at $\mathbf{g}^*$.

Defining, on each edge $k = 0, 1, \dots, N-1$, the *Lie algebra Hessian blocks*

$$\begin{aligned} \tilde{\mathcal{A}}_k &= (T_e L_{g_k})^* D_{11} L_d(g_k, g_{k+1}) (T_e L_{g_k}), \\ \tilde{\mathcal{B}}_k &= (T_e L_{g_{k+1}})^* D_{22} L_d(g_k, g_{k+1}) (T_e L_{g_{k+1}}), \\ \tilde{\mathcal{C}}_k &= (T_e L_{g_k})^* D_{12} L_d(g_k, g_{k+1}) (T_e L_{g_{k+1}}), \end{aligned} \quad (5.9)$$

we have $\tilde{\mathcal{D}}_k = \tilde{\mathcal{B}}_{k-1} + \tilde{\mathcal{A}}_k$ at $\mathbf{g}^*$, just as in (3.9) and (3.11). Putting it all together, the full Lie algebra Hessian at $\mathbf{g}^*$ is a familiar-looking block-tridiagonal matrix

$$\tilde{H}(\mathbf{g}^*) = \begin{pmatrix} \tilde{\mathcal{D}}_1 & \tilde{\mathcal{C}}_1 & & & \\ \tilde{\mathcal{C}}_1^T & \tilde{\mathcal{D}}_2 & \tilde{\mathcal{C}}_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \tilde{\mathcal{C}}_{N-3}^T & \tilde{\mathcal{D}}_{N-2} & \tilde{\mathcal{C}}_{N-2} \\ & & & \tilde{\mathcal{C}}_{N-2}^T & \tilde{\mathcal{D}}_{N-1} \end{pmatrix}. \quad (5.10)$$

If $G = \mathbb{R}^n$, then $T_e L_g = (T_e L_g)^* = \text{Id}$ and so (5.9) reduce to (3.9) and (3.11); and (5.10) to (3.8), recovering the result obtained in vector spaces, not only in structure but in value.

5.2.2. Convergence criteria

The Jacobi-Newton-G iteration is a smooth map $T : G^{N-1} \rightarrow G^{N-1}$ with $\mathbf{g}^*$ as a fixed point. To apply classical fixed-point theory, we move T to the Lie algebra chart based on a perturbation ξ from the fixed point $\mathbf{g}^*$

$$\begin{aligned} \Psi : V \subset \mathfrak{g}^{N-1} &\rightarrow U \subset G^{N-1} \\ \Psi(\xi) &= (g_1^* \tau(\xi_1), \dots, g_{N-1}^* \tau(\xi_{N-1})), \end{aligned}$$

defined on a neighborhood V of 0 where τ is a local diffeomorphism on $\mathfrak{g}$. The perturbation is then updated with $\tilde{T} = \Psi^{-1} \circ T \circ \Psi : V \rightarrow \mathfrak{g}^{N-1}$, satisfying $\tilde{T}(0) = 0$.

Lemma 8 (Derivative of the perturbation iteration). *Suppose each diagonal block $\tilde{\mathcal{D}}_k(\mathbf{g}^*)$ is positive definite. Then $\tilde{T}'(0) = \tilde{J}$, where $\tilde{H}(\mathbf{g}^*) = \tilde{D} + \tilde{L} + \tilde{U}$ is the decomposition into block-diagonal, strictly lower block-triangular, and strictly upper block-triangular parts, and $\tilde{J} = -\tilde{D}^{-1}(\tilde{L} + \tilde{U})$ is the associated block Jacobi iteration matrix.*

Proof. Let $\tilde{T} = (\tilde{T}_1, \dots, \tilde{T}_{N-1})$ where each $\tilde{T}_k : V \rightarrow \mathfrak{g}$ is the local perturbation update. Given the perturbed path $\mathbf{g} = \Psi(\xi)$, the Jacobi-Newton-G step produces corrections $\delta\xi_k(\xi) = -\tilde{\mathcal{D}}_k(\mathbf{g})^{-1} \tilde{r}_k(\mathbf{g})$ and updates $\bar{g}_k = g_k \tau(\delta\xi_k)$. The new chart coordinate of $\bar{g}_k$ follows from Ψ :

$$\begin{aligned}\bar{g}_k &= g_k \tau(\delta\xi_k) = g_k^* \tau(\xi_k) \tau(\delta\xi_k), \\ \bar{\xi}_k &= \tau^{-1}(\tau(\xi_k) \tau(\delta\xi_k)),\end{aligned}$$

so

$$\bar{\xi}_k = \tilde{T}_k(\xi) = \mu(\xi_k, \delta\xi_k(\xi)), \quad \mu(\alpha, \beta) = \tau^{-1}(\tau(\alpha) \tau(\beta)). \quad (5.11)$$

At $\xi = 0$: $\mathbf{g} = \mathbf{g}^*$, so $r_k(\mathbf{g}^*, g_k^*) = 0$, hence $\tilde{r}_k(\mathbf{g}^*) = 0$, $\delta\xi_k(0) = 0$ (note that the update exists because $\tilde{\mathcal{D}}_k$ is invertible by hypothesis), and $\tilde{T}_k(0) = \mu(0, 0) = 0$.

Since $\tau(0) = e$ and $T_0\tau = \text{Id}_{\mathfrak{g}}$, differentiating $\mu(\alpha, \beta) = \tau^{-1}(\tau(\alpha)\tau(\beta))$ at $(0, 0)$ gives $\partial_\alpha \mu(0, 0) = \partial_\beta \mu(0, 0) = \text{Id}_{\mathfrak{g}}$. Differentiating (5.11) at $\xi = 0$ and using $\delta\xi_k(0) = 0$,

$$\partial_{\xi_j} \tilde{T}_k \Big|_0 = \delta_{kj} \text{Id}_{\mathfrak{g}} + \partial_{\xi_j} \delta\xi_k \Big|_0. \quad (5.12)$$

The product rule applied to $\delta\xi_k(\xi) = -\tilde{\mathcal{D}}_k(\mathbf{g})^{-1} \tilde{r}_k(\mathbf{g})$ gives, at $\xi = 0$,

$$\begin{aligned}\partial_{\xi_j} \delta\xi_k \Big|_0 &= - \underbrace{\partial_{\xi_j} \tilde{\mathcal{D}}_k^{-1} \Big|_0}_{= 0 \text{ since } \tilde{r}_k(\mathbf{g}^*)=0} \tilde{r}_k(\mathbf{g}^*) - \tilde{\mathcal{D}}_k(\mathbf{g}^*)^{-1} \partial_{\xi_j} \tilde{r}_k \Big|_0.\end{aligned}$$

Differentiating $\tilde{r}_k(\mathbf{g})[\eta] = r_k(\mathbf{g}, g_k)[T_e L_{g_k} \eta]$ in ξ_j results in

$$\partial_{\xi_j} \tilde{r}_k \Big|_0 \eta = \underbrace{\partial_{\xi_j} r_k \Big|_0 (T_e L_{g_k^*} \eta)}_{\text{main term}} + \underbrace{r_k(\mathbf{g}^*, g_k^*) \left(\partial_{\xi_j} (T_e L_{g_k}) \Big|_0 \eta \right)}_{\text{variation-of-pullback}} = 0$$

where the variation-of-pullback term is 0 due to $r_k(\mathbf{g}^*, g_k^*) = 0$, so only the main term survives:

$$\partial_{\xi_j} \tilde{r}_k \Big|_0 = (T_e L_{g_k^*})^* \partial_{\xi_j} r_k \Big|_0.$$

We can now identify this expression with the gradient (5.6), which has a similar form $\partial_{\xi_k} \Sigma(\xi) = \Phi_k(\xi_k)^* r_k$, but with the chart frame Φ_k in place of the left translation. Its derivative has again the pattern of a term $\Phi_k(0) = T_e L_{g_k^*}$ multiplied by the derivative of the residual, and a variation term weighted by $r_k(\mathbf{g}^*, g_k^*) = 0$ that disappears. This leaves us with the same expression as above. That is,

$$\partial_{\xi_j} \tilde{r}_k \Big|_0 = \partial_{\xi_j} \partial_{\xi_k} \Sigma(0) = \tilde{H}_{kj},$$

the (k, j) block of the Lie algebra Hessian, whose value was computed in section 5.2.1: for $j \in \{k-1, k, k+1\}$,

$$\begin{aligned}\partial_{\xi_{k-1}} \tilde{r}_k|_0 &= \tilde{C}_{k-1}^T, \\ \partial_{\xi_k} \tilde{r}_k|_0 &= \tilde{\mathcal{D}}_k, \\ \partial_{\xi_{k+1}} \tilde{r}_k|_0 &= \tilde{C}_k,\end{aligned}$$

and 0 otherwise. Substituting into (5.12),

$$\tilde{T}'(0) = I - \tilde{D}^{-1} \tilde{H}(\mathbf{g}^*) = -\tilde{D}^{-1}(\tilde{L} + \tilde{U}) = \tilde{J}. \quad \square$$

Remark 5 (Where the manifold structure enters). The Lie group G enters the convergence story only through the chart Ψ and the Lie algebra Hessian blocks of section 5.2.1. Lemma 8 packages this geometric content into a single linear-algebraic identity, $\tilde{T}'(0) = \tilde{J}$. From that point on, $\tilde{T}$ is just a smooth map on $\mathfrak{g}^{N-1} \cong \mathbb{R}^{(N-1)\dim \mathfrak{g}}$, and the remaining work is, effectively, classical analysis in $\mathbb{R}^n$.

The following theorem is essentially the same as theorem 1, but the proof is slightly different because we cannot directly apply lemma 4 (since that talks about Jacobi-Newton applied to vector spaces). The main difference is that we use [11, Thm. 10.1.3] which assumes less about how the iteration procedure is set up, and is the reason we needed lemma 8. Regardless, the proof is brief and similar in spirit to the one in [1, Prop. 4.5].

Theorem 9 (Lie group convergence). *Let $\mathbf{g}^*$ be a solution of the DEL equations. If the Lie algebra Hessian $\tilde{H}(\mathbf{g}^*)$ is positive definite, then the Jacobi-Newton-G method converges locally to $\mathbf{g}^*$.*

Proof. The diagonal blocks $\tilde{\mathcal{D}}_k(\mathbf{g}^*)$ are principal blocks of the positive-definite symmetric matrix $\tilde{H}(\mathbf{g}^*)$, hence positive definite (by Sylvester's criterion and a permutation argument). In particular, $\tilde{D}(\mathbf{g}^*)$ is nonsingular, so $\tilde{J} = -\tilde{D}^{-1}(\tilde{L} + \tilde{U})$ is well-defined. Since $\tilde{H}(\mathbf{g}^*)$ is symmetric, positive definite, and block-tridiagonal, [12, Thm. 6.38] gives $\rho(\tilde{J}) < 1$.

By lemma 8, $\tilde{T}'(0) = \tilde{J}$, so the smooth fixed-point map $\tilde{T}$ on $\mathfrak{g}^{N-1}$ satisfies $\tilde{T}(0) = 0$ and $\rho(\tilde{T}'(0)) < 1$. Ostrowski's theorem [11, Thm. 10.1.3] yields an open neighborhood $V_0 \ni 0$ in $\mathfrak{g}^{N-1}$ such that $\tilde{T}^a(\xi) \rightarrow 0$ for all $\xi \in V_0$; since Ψ is a local diffeomorphism at 0, there is a convergent neighborhood given by $\mathcal{D} = \Psi(V_0)$. $\square$

Just as in the vector space case (theorem 2), the global positive-definiteness check on $\tilde{H}$ reduces to per-edge checks on the local 2×2 -block Hessians

$$\tilde{H}_k = \begin{pmatrix} \tilde{\mathcal{A}}_k & \tilde{\mathcal{C}}_k \\ \tilde{\mathcal{C}}_k^T & \tilde{\mathcal{B}}_k \end{pmatrix}.$$

Remark 6 (Local Lie algebra Hessian condition). If $\tilde{H}_k$ is positive definite for every edge $k = 0, 1, \dots, N-1$, then $\tilde{H}(\mathbf{g}^*)$ is positive definite, and by theorem 9 the Jacobi-Newton-G method converges locally to $\mathbf{g}^*$. The argument is identical to that of theorem 2, since $\tilde{H}$ has the same block-tridiagonal structure (remark 5).

Remark 7 (Lie algebra Hessian positive definiteness criterion). By remark 5, $\tilde{H}(\mathbf{g}^*)$ is a symmetric block-tridiagonal matrix on $\mathfrak{g}^{N-1} \cong \mathbb{R}^{(N-1)\dim \mathfrak{g}}$, so the necessary and sufficient criterion of theorem 3 applies verbatim: $\tilde{H}(\mathbf{g}^*)$ is positive definite if and only if the matrices $\tilde{\Lambda}_1 = \tilde{\mathcal{D}}_1$ and

$$\tilde{\Lambda}_i = \tilde{\mathcal{D}}_i - \tilde{\mathcal{C}}_{i-1}^T \tilde{\Lambda}_{i-1}^{-1} \tilde{\mathcal{C}}_{i-1}, \quad 2 \leq i \leq N-1,$$

are all positive definite, in which case the Jacobi-Newton-G method converges locally to $\mathbf{g}^*$ by theorem 9.

The blocks $\tilde{\mathcal{A}}_k, \tilde{\mathcal{B}}_k, \tilde{\mathcal{C}}_k, \tilde{\mathcal{D}}_k$ depend on the choice of trivialization (left vs. right) and retraction τ , but positive definiteness of $\tilde{H}(\mathbf{g}^*)$ does not: a change of chart at $\mathbf{g}^*$ acts on $\tilde{H}$ by congruence with a block-diagonal invertible matrix, which preserves the sign of the spectrum.

Remark 8 (Connection to the discrete Euler-Poincaré equations). Many mechanical systems on Lie groups are *left-invariant* (or right-invariant), meaning that $L_d(g_k, g_{k+1}) = L_d(hg_k, hg_{k+1})$ for all $h \in G$. If this is the case, the Lagrangian factors through the quotient $f_k = g_k^{-1}g_{k+1}$, resulting in a *reduced Lagrangian* $\ell : G \rightarrow \mathbb{R}$ with $\ell(f_k) = L_d(g_k, g_{k+1})$. Then, the DEL equations on G are equivalent to the *discrete Euler-Poincaré (DEP)* equations [9, §3.4]

$$(T_e L_{f_{k-1}})^* d\ell(f_{k-1}) = (T_e R_{f_k})^* d\ell(f_k). \quad (\text{DEP})$$

Solutions of the DEP iteration on quotients $\{f_k\}$ is equivalent to solutions of the DEL iteration on $\{g_k\}$ [13, Thm. 2.1]. Consequently, theorem 9 can equivalently be stated for the DEP iteration. The non-reduced formulation we use is more general, and for invariant systems simplifies the convergence analysis at the cost of a slightly larger state space.

5.3. Implementation and examples

Implementing this extension in practice is not completely straightforward if one does not use some library that provides support for these operations. For our implementation, we made the algorithm work given a `Manifold` interface, which requires definitions of:

1. the numerical representation of elements of G
2. the pullbacked differential operator, $(T_e L_g)^* T_g : (G \rightarrow \mathbb{R}) \rightarrow G \rightarrow \mathfrak{g}^*$
3. the group action, $\cdot : G \times G \rightarrow G$

4. a retraction, $\tau : \mathfrak{g} \rightarrow G$

Elements of the tangent and cotangent space are represented by their trivialized counterparts in the algebra, and elements of the algebra (and coalgebra) are represented via the $\mathbb{R}^n$ isomorphism we know must exist. For instance, on $G = \text{SO}(3)$, elements of $\mathfrak{g} = \mathfrak{so}(3)$ are skew-symmetric 3×3 matrices that have an isomorphism with $\mathbb{R}^3$ via the hat map:

$$\hat{\xi} = \widehat{\begin{bmatrix} x \\ y \\ z \end{bmatrix}} = \begin{bmatrix} 0 & -z & y \\ z & 0 & -x \\ -y & x & 0 \end{bmatrix}.$$

The only thing missing is a way to calculate the second derivative, but this can actually be calculated with no additional knowledge, given that we are representing $\mathfrak{g}$ and $\mathfrak{g}^*$ as vectors. Namely, we differentiate directly the residual as a function of the algebra, by taking directly the Jacobian of (5.2). Since we represent $\mathfrak{g}$ and $\mathfrak{g}^*$ as $\mathbb{R}^n$, this can be done using any autodifferentiation library (such as JAX).

This is enough to implement the algorithm, although in practice we also ask for a way to construct a tangent “velocity” vector as $\text{tangent}(q_0, q_1, h)$ which we use to be able to define just the continuous Lagrangian and discretize it automatically.

The source code, including all the code and parameter setup for the following examples, is available in the repository at <https://github.com/sebajf/pdel-mechanics> (GitHub) and <https://git-archive.odilf.com/odilf/pdel-mechanics> (archived snapshot).

5.3.1. Rigid body satellite positioning

Consider a satellite at some position that we want to get to some other point at a specific time with a specific orientation (to, say, study an event in the surface of a star).

The trajectory of the satellite is affected by the gravitational pull of nearby bodies in space. To find what trajectories are possible, one could try to just model the path in $\mathbb{R}^3$ and use a constant angular velocity to re-orient the satellite. However, the pull from nearby bodies can change how the satellite rotates, and the experienced pull in turn changes depending on the orientation.

A more accurate way of modeling this situation is by using $\text{SE}(3)$, the group of rigid body motions, which automatically handles this complexity in its manifold geometry. We model the satellite as a finite collection of body-frame points $\{r_i^b\}$ with masses m_i , so the world-frame position of point i is $x + R r_i^b$. For J celestial bodies at positions x_j with

gravitational parameters $\mu_j = GM_j$, the potential and kinetic energies are

$$V(R, x) = - \sum_{i,j} \frac{\mu_j m_i}{\|x + R r_i^b - x_j\|},$$

$$T(\omega, v) = \frac{1}{2} \omega^T I \omega + \frac{1}{2} m \|v\|^2,$$

where I is the body-frame inertia tensor, $\omega \in \mathbb{R}^3$ is the body-frame angular velocity and $v \in \mathbb{R}^3$ is the world-frame translational velocity (this mixed convention is because of the discretization: the angular part is $\log(R_k^T R_{k+1})$, hence body-frame; while the translational part is $x_{k+1} - x_k$, hence world-frame). The Lagrangian is $L = T - V$.

We discretize with the trapezoidal quadrature. With timestep $h = T/N$, each edge carries the finite-difference velocity $\xi_k = (\omega_k, v_k) \in \mathfrak{se}(3)$,

$$\omega_k = \frac{1}{h} \log(R_k^T R_{k+1}), \quad v_k = \frac{1}{h} (x_{k+1} - x_k),$$

whose angular part is body-frame and translational part world-frame, and we set

$$L_d(g_k, g_{k+1}) = \frac{h}{2} (L(g_k, \xi_k) + L(g_{k+1}, \xi_k)).$$

The resulting trajectory of a prism-like satellite under interesting conditions is shown in fig. 5.1 (full parameter setup³ in the source code).

We see in the front view (fig. 5.1b) that even though the satellite only has to increase its height by a small amount, the physical trajectory needs to take a detour and make a bigger arc movement. The simulation shows us that a direct approach does not seem to allow reaching the final desired orientation, as the pull from the two bodies would rotate the satellite. In fact, in the top view (fig. 5.1c) we can see that the path taken in the end does a smooth approach from above, where the pull from both bodies seems to be balanced in such a way that the satellite can rendezvous with the target orientation.

5.3.2. Zermelo fuel-optimal navigation in SE(3)

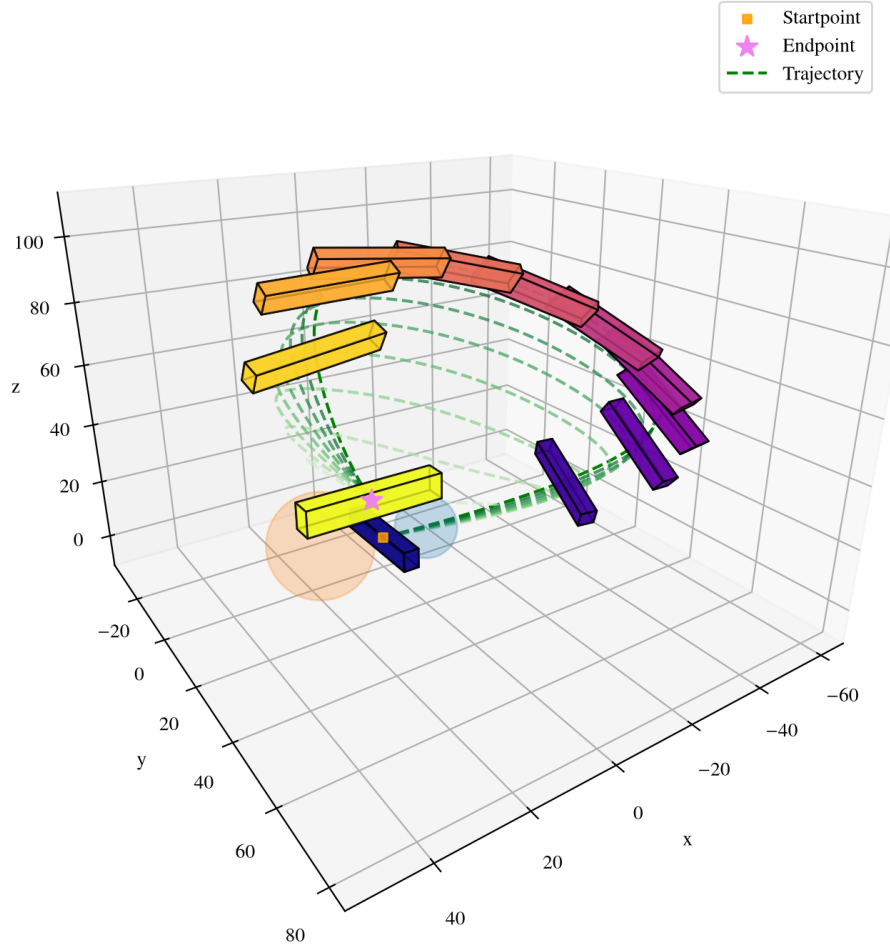
Lagrangian formulations can also be used to find optimal solutions to certain problems. In [1, Example 5.2] they formulate a fuel-optimal navigation problem in a wind field. We extend it to SE(3) by letting the system take into account the orientation of the ship, which is in fact a spaceship in our model that can reorient itself in three dimensions.

We model the ship as a box with side lengths (a, b, c) and one long axis $(a \gg b, c)$. The cost of moving is the sum of two quadratic forms. The first is a *turning cost* that penalizes spinning,

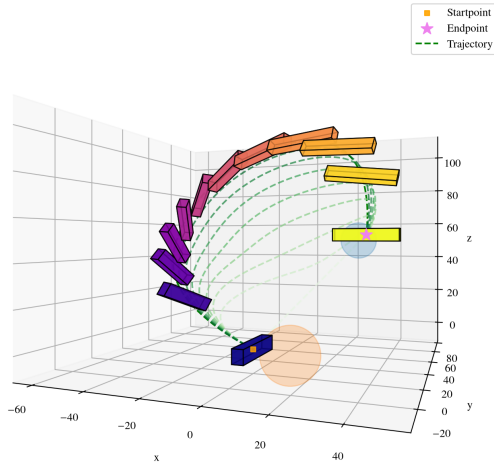
$$\frac{1}{2} \omega^T I \omega, \quad I = \text{diag}(b^2 + c^2, a^2 + c^2, a^2 + b^2),$$

where I is the body-frame inertia of the box. The second is a *drag cost* that penalizes moving against the flow $W(x)$. The resistance along each body axis is proportional to the

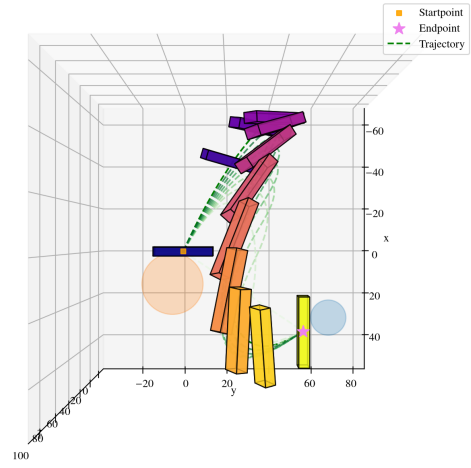
³The relative size of the satellite to the celestial bodies is exaggerated to make the rotational effects easier to see. One could think of it as a very big spaceship, or as very small planets/asteroids/comets.



(a) Diagonal view.



(b) Front view.



(c) Top view.

Fig. 5.1. Satellite physical simulation. Satellite starts at lower z needs to reach the position and orientation at higher z . Prisms indicate evenly spaced snapshots of the satellite's position and orientation. Green path indicates positional trajectory, fainter green paths indicate intermediate solver iterations. Circles indicate celestial bodies that affect the satellite, size indicating pulling strength (i.e., mass).

area of the face perpendicular to it, so the drag weights are those face areas,

$$D = k \text{diag}(bc, ac, ab),$$

where k is the drag coefficient. The long axis gets the smallest weight bc , so the ship slips most easily when travelling lengthwise.

Since turning also drags the ship's corners through the flow, the drag is evaluated not at the center but at each of the eight corners r_c of the box. The velocity of corner r_c relative to the flow, expressed in the body frame, is

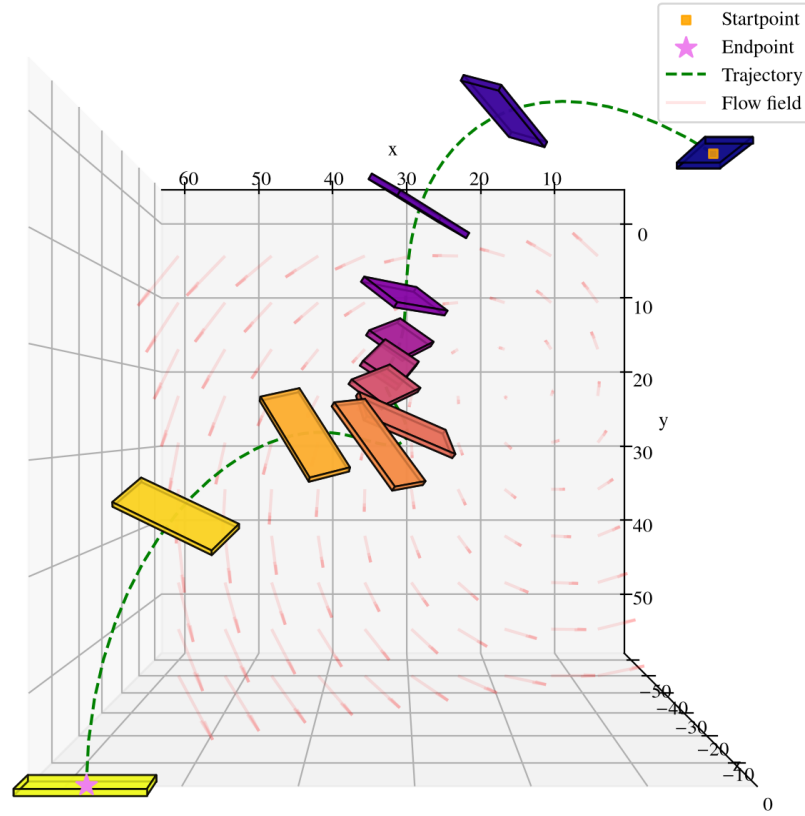
$$s_c = R^T(v - W(x)) + \omega \times r_c,$$

the world-frame slip $v - W(x)$ rotated into the body frame by R^T , plus the rotational contribution $\omega \times r_c$ coming from the corner's offset. Averaging over the 8 corners gives the Lagrangian

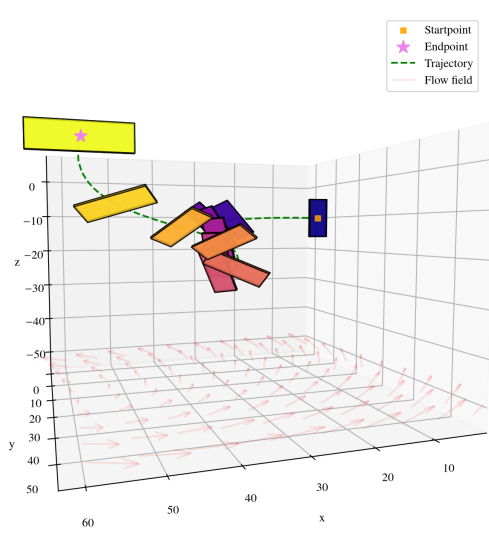
$$L(R, x, \omega, v) = \frac{1}{2} \omega^T I \omega + \frac{1}{16} \sum_c s_c^T D s_c.$$

We discretize with the trapezoidal quadrature, exactly as in the satellite case.

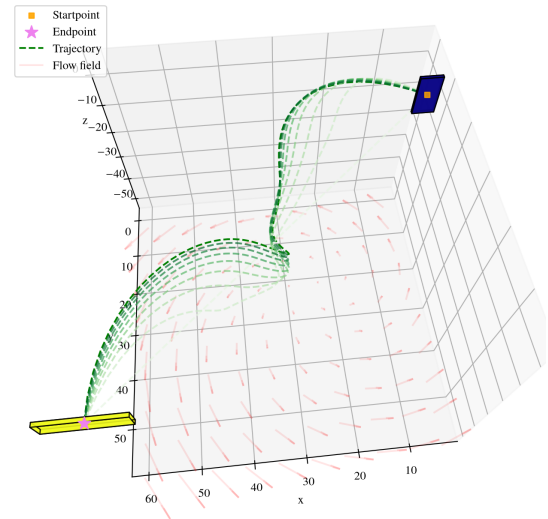
The simulation results are presented in fig. 5.2. We see in the top view (fig. 5.2a) that the ship does not head straight for the target but rather follows two arcs, continually turning so that its long, low-drag axis stays aligned with the slip $v - W(x)$. Midway through, the snapshots bunch tightly together at the center of the vortex where the wind is small. The ship slows almost to a standstill, and the drag is cheap regardless of orientation, so it uses this stretch to set up the final rotation. In the side view (fig. 5.2b) we can then see it re-align and accelerate out into the final approach, orientating itself to take advantage of the slight upwards wind, reaching the target position and orientation.



(a) Top view.



(b) Side view.



(c) Positional trajectory only. Fainter green paths indicate intermediate solver iterations.

Fig. 5.2. Zermelo fuel-optimal navigation in $SE(3)$. Ship starts at the orange square and needs to reach the position and orientation at the violet star. Prisms indicate evenly spaced snapshots of the ship's position and orientation. Green path indicates positional trajectory. Red arrows indicate the wind field, the blue arrows indicate the slip at the ship's current position.

6. CONCLUSIONS AND FUTURE WORK

This thesis set out to extend the parallel Jacobi-Newton algorithm of [1] beyond the unforced vector space setting where it was originally formulated, and carry out the respective convergence analyses. Two extensions were carried out: one to forced (Lagrange-d'Alembert) systems in chapter 4, and one to systems whose configuration manifold is a Lie group in chapter 5. Sufficient conditions for local convergence (theorems 5, 6 and 9) were found, and the Lie group case inherits the easy-to-compute condition given in theorem 3.

The underlying observation of the presented work is that the idea of the parallel algorithm does not fundamentally depend on Q being a vector space, nor on the existence of an action that is being extremized. What it depends on is that the discrete equations of motion be a system of local couplings between adjacent nodes, so that the Jacobian of the residual is block-tridiagonal. This locality is derived from the variational discretization (writing the action as a sum $\sum_k L_d(q_k, q_{k+1})$ over edges) and is preserved by both forcing and by left-trivialization to the Lie algebra. Through this lens, the role played by the vector space hypothesis and by the variational hypothesis becomes clear, and the two extensions can be read as decoupling the algorithm from each in turn.

Chapter 4 drops the variational hypothesis. The algorithm is trivially adaptable, but the discrete Lagrange-d'Alembert principle is not, in general, the extremization of any functional, so the residual $\hat{R}$ is no longer a gradient and the matrix $H^f = \nabla \hat{R}$ is no longer a Hessian. Concretely, this means H^f is not symmetric, and the previously used convergence criteria based on theorem 1 are no longer applicable. What survives is the block-tridiagonal structure of H^f , which is what lets us prove the two structural criteria of section 4.2: a local condition (theorem 5) that generalizes the unforced local-Hessian criterion, and a perturbation result (theorem 6) that treats the forced case as a continuation of the unforced one, with an explicit smallness bound on the forces.

Chapter 5 drops the vector space hypothesis. Here, the algorithm itself has to be adapted, computing updates in the Lie algebra and applying them via retractions $\tau : \mathfrak{g} \rightarrow G$. For the convergence analysis, the residual is pulled back to the Lie algebra via left-translation and the lifted action on $\mathfrak{g}^{N-1}$ has a (true) Hessian $\tilde{H}$ with the same block-tridiagonal structure as H . The iteration, read in a Lie algebra chart at the solution, is similar to a block Jacobi-Newton iteration on a vector space. Theorem 9 therefore looks identical to the vector space theorem, and theorem 3 applies to $\tilde{H}$ without modification. The work is in showing that “read in a chart at the solution” is the right thing to do (lemma 8) and in translating the local Hessian blocks into Lie algebra coordinates (section 5.2.1); once that is set up, the algebraic content of the proofs is the same.

Both extensions were implemented and validated on numerical examples. Taken together, these two extensions cover many of the systems that motivate variational integrators in practice, such as dissipative systems and rigid bodies. The criteria obtained are suffi-

cient but not necessary, and the forced and Lie group cases were treated independently rather than in combination, which are some ways that the presented work can be expanded upon.

6.1. Future work

The contributions of this thesis suggest several natural continuations, which we group into three themes: strengthening the convergence theory, extending the parallel algorithm to even more kinds of Lagrangian systems, and transitive improvements to the underlying parallel algorithm explored.

Convergence theory

The most immediate missing piece is an analog of theorem 3 for forced systems. I.e., a quick way to check in general if the algorithm will converge for some forced Lagrangian. The difficulty comes from the fact that there is no clear way to adapt theorem 3 to forced systems, since we do not have a “global” criterion as being positive definite is for unforced systems. The two most straightforward approaches to solve this are to either prove the local condition (theorem 5) globally and find a quick way to compute it there, or to directly find a local condition similar to theorem 5 that is less restrictive.

As an alternative approach to the analysis of forced systems in general, it might be possible to apply Galley’s idea [14] [9, §§4.2.1, 4.3.3], which reformulates forced systems as unforced, constrained systems in a configuration space of twice the dimension, essentially modeling, for each variable, an “environment” counterpart that absorbs the force. A cursory attempt at applying this idea to a simple dissipative example resulted in conditions that were much harder to work with than the ones obtained from an equivalent, unforced, time-dependent Lagrangian. However, this method is known to lead to elegant results (see, for instance, [15]), and it might be possible to get usable criteria if one is careful with the constraining.

A separate loose end is that forced systems and Lie groups were treated independently in chapter 4 and chapter 5. At an implementation level, the combination is straightforward, but the convergence analysis is not as much. The forcing contributes additional cross terms to the local Hessian, and it is not directly clear whether all curvature of the retraction and its derivatives will disappear at the points of interest as in the unforced case. However, it seems possible that the results from section 4.2 can be adapted to work on Lie groups following a similar line of reasoning, perhaps with some additional work.

Further extensions

The other direction of future work is on extending the algorithm to work with even more kinds of Lagrangian systems. Probably the main interest is the extension to *constrained systems*, such as the rigid links of a robotic arm (holonomic) or rolling contact of a wheel on a surface (nonholonomic). Variational integrators for such systems have been studied in the serial setting [4, §§3.4–3.5] [9, §5.4], but the interaction between the constraint structure and the parallel Jacobi-Newton update is non-trivial: the residual now lives on the constrained submanifold, and the local Hessian must be projected accordingly.

In terms of configuration space, Lie groups are less restrictive than vector spaces, but the structure could be loosened even more. For instance, one could consider *homogeneous groups* G/H , where the symmetries are provided by a group external to the manifold. The sphere, $S^n = \mathrm{SO}(n+1)/\mathrm{SO}(n)$, is an example of such a manifold with physical relevance. Another loosening can be done in the form of *Lie groupoids*, groupoids $G \rightrightarrows M$ where G and M are manifolds, which also model real physical systems [16].

Refinements of the parallel algorithm

Finally, it might also be possible to do orthogonal improvements to the parallel algorithm. An example could be adaptive time-stepping (i.e., using a variable timestep h), which by default is known to break symplecticity but it might be able to be mitigated with some modifications. Such improvements could, then, be applied to the extensions, either automatically or with some additional work, depending on the nature of the improvement.

BIBLIOGRAPHY

- [1] S. J. Ferraro, D. Martín De Diego, and R. T. Sato Martín De Almagro, “A parallel iterative method for variational integration,” *Journal of Computational Dynamics*, vol. 12, no. 4, pp. 528–553, 2025. doi: [10.3934/jcd.2025006](https://doi.org/10.3934/jcd.2025006). Accessed: Jan. 2, 2026. [Online]. Available: <https://www.aims sciences.org//article/doi/10.3934/jcd.2025006>.
- [2] J. M. Lee, “Smooth Manifolds,” in *Introduction to Smooth Manifolds*. New York, NY: Springer New York, 2013, vol. 218, pp. 1–31. doi: [10.1007/978-1-4419-9982-5_1](https://doi.org/10.1007/978-1-4419-9982-5_1). Accessed: Jun. 16, 2026. [Online]. Available: https://link.springer.com/10.1007/978-1-4419-9982-5_1.
- [3] J. E. Marsden and T. S. Ratiu, *Introduction to Mechanics and Symmetry: A Basic Exposition of Classical Mechanical Systems* (Texts in Applied Mathematics), J. E. Marsden, L. Sirovich, M. Golubitsky, and W. Jäger, red. New York, NY: Springer New York, 1999, vol. 17. doi: [10.1007/978-0-387-21792-5](https://doi.org/10.1007/978-0-387-21792-5). Accessed: Apr. 18, 2026. [Online]. Available: <http://link.springer.com/10.1007/978-0-387-21792-5>.
- [4] J. E. Marsden and M. West, “Discrete mechanics and variational integrators,” *Acta Numerica*, vol. 10, pp. 357–514, May 2001. doi: [10.1017/S096249290100006X](https://doi.org/10.1017/S096249290100006X). Accessed: Apr. 16, 2026. [Online]. Available: https://www.cambridge.org/core/product/identifier/S096249290100006X/type/journal_article.
- [5] F. Gay-Balmaz, D. D. Holm, D. M. Meier, T. S. Ratiu, and F.-X. Vialard, “Invariant higher-order variational problems II,” *Journal of Nonlinear Science*, vol. 22, no. 4, pp. 553–597, Aug. 2012. doi: [10.1007/s00332-012-9137-2](https://doi.org/10.1007/s00332-012-9137-2). arXiv: [1112.6380](https://arxiv.org/abs/1112.6380) [math]. Accessed: Apr. 28, 2026. [Online]. Available: <http://arxiv.org/abs/1112.6380>.
- [6] H. Munthe-Kaas, “Runge-Kutta methods on Lie groups,” *BIT Numerical Mathematics*, vol. 38, no. 1, pp. 92–111, Mar. 1998. doi: [10.1007/BF02510919](https://doi.org/10.1007/BF02510919). Accessed: Jun. 17, 2026. [Online]. Available: <http://link.springer.com/10.1007/BF02510919>.
- [7] E. Hairer, G. Wanner, and C. Lubich, *Geometric Numerical Integration* (Springer Series in Computational Mathematics). Berlin/Heidelberg: Springer-Verlag, 2006, vol. 31. doi: [10.1007/3-540-30666-8](https://doi.org/10.1007/3-540-30666-8). Accessed: Jun. 21, 2026. [Online]. Available: <http://link.springer.com/10.1007/3-540-30666-8>.
- [8] G. Zhong and J. E. Marsden, “Lie-Poisson Hamilton-Jacobi theory and Lie-Poisson integrators,” *Physics Letters A*, vol. 133, no. 3, pp. 134–139, Nov. 1988. doi: [10.1016/0375-9601\(88\)90773-6](https://doi.org/10.1016/0375-9601(88)90773-6). Accessed: Jun. 21, 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/0375960188907736>.

- [9] R. T. Sato Martín de Almagro, “Discrete Mechanics for Forced and Constrained Systems,” Universidad Complutense de Madrid, Madrid, 2020, 223 pp. [Online]. Available: <https://eprints.ucm.es/id/eprint/59473/>.
- [10] R. A. Horn and C. R. Johnson, *Matrix Analysis*, 2nd ed. Cambridge: Cambridge University Press, 2012.
- [11] J. M. Ortega and W. C. Rheinboldt, *Iterative Solution of Nonlinear Equations in Several Variables* (Computer Science and Applied Mathematics). San Diego: Academic press, 1970.
- [12] O. Axelsson, *Iterative Solution Methods*, 1st ed. Cambridge University Press, Mar. 25, 1994. doi: [10.1017 / CB09780511624100](https://doi.org/10.1017/CB09780511624100). Accessed: Jun. 6, 2026. [Online]. Available: <https://www.cambridge.org/core/product/identifier/9780511624100/type/book>.
- [13] J. E. Marsden, S. Pekarsky, and S. Shkoller. “Discrete Euler-Poincaré and Lie-Poisson Equations.” version 1, Accessed: Apr. 23, 2026. [Online]. Available: <https://arxiv.org/abs/math/9909099>, pre-published.
- [14] C. R. Galley, “Classical Mechanics of Nonconservative Systems,” *Physical Review Letters*, vol. 110, no. 17, p. 174301, Apr. 22, 2013. doi: [10.1103/PhysRevLett.110.174301](https://doi.org/10.1103/PhysRevLett.110.174301). Accessed: Jun. 17, 2026. [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevLett.110.174301>.
- [15] D. M. de Diego and R. S. M. de Almagro, “Variational order for forced Lagrangian systems,” *Nonlinearity*, vol. 31, no. 8, pp. 3814–3846, Aug. 1, 2018. doi: [10.1088 / 1361-6544/aac5a6](https://doi.org/10.1088/1361-6544/aac5a6). arXiv: [1712.09377](https://arxiv.org/abs/1712.09377) [math-ph]. Accessed: Jun. 17, 2026. [Online]. Available: <http://arxiv.org/abs/1712.09377>.
- [16] A. Weinstein. “Groupoids: Unifying internal and external symmetry.” arXiv: [math / 9602220](https://arxiv.org/abs/math/9602220), Accessed: Jun. 22, 2026. [Online]. Available: <http://arxiv.org/abs/math/9602220>, pre-published.

APPENDIX A. THE LOCAL CONDITION IN THE SCALAR TWO-NODE CASE

In section 4.2.1 we considered the condition

$$\lambda_{\min}(\text{Sym}(H^f)) > \|\text{Skew}(H^f)\|_2,$$

which confines the eigenvalues of H^f to a 45° sector in the right half-plane and is a natural generalization of $H^f \succ 0$ to the non-symmetric setting. We claimed there that it implies $\rho(J^f) < 1$ in the scalar case, but that the general block case remains open. Here we prove the scalar case.

Lemma 10 (Scalar local condition). *Let $n = 1$ and $N = 3$, so that H^f is a 2×2 matrix with positive diagonal (two interior nodes). Then*

$$\lambda_{\min}(\text{Sym}(H^f)) > \|\text{Skew}(H^f)\|_2 \implies \rho(J^f) < 1.$$

Proof. Let the entries of the forced Hessian be

$$H^f = \begin{pmatrix} d_1 & c^+ \\ c^- & d_2 \end{pmatrix}, \quad d_1, d_2 > 0,$$

where d_1, d_2 are the scalar diagonal blocks $\hat{\mathcal{D}}_1, \hat{\mathcal{D}}_2$ and c^+, c^- the cross terms $\hat{\mathcal{C}}_1^+, \hat{\mathcal{C}}_1^-$ of (4.1). The Jacobi matrix is then

$$J^f = -\text{diag}(d_1, d_2)^{-1} \left(H^f - \text{diag}(d_1, d_2) \right) = \begin{pmatrix} 0 & -c^+/d_1 \\ -c^-/d_2 & 0 \end{pmatrix},$$

with characteristic polynomial $\lambda^2 - c^+c^-/(d_1d_2)$. Hence $\rho(J^f)^2 = |c^+c^-|/(d_1d_2)$, and the claim $\rho(J^f) < 1$ amounts to showing $|c^+c^-| < d_1d_2$.

To separate the symmetric and skew parts of H^f , let $s = (c^+ + c^-)/2$ and $a = (c^+ - c^-)/2$, so that $c^+c^- = s^2 - a^2$ and

$$\text{Sym}(H^f) = \begin{pmatrix} d_1 & s \\ s & d_2 \end{pmatrix}, \quad \|\text{Skew}(H^f)\|_2 = |a|.$$

The eigenvalues of $\text{Sym}(H^f)$ are given by $\lambda^2 - (d_1 + d_2)\lambda + (d_1d_2 - s^2) = 0$, so

$$\begin{aligned} \lambda_{\min}(\text{Sym}(H^f)) &= \frac{d_1 + d_2}{2} - \sqrt{\left(\frac{d_1 + d_2}{2}\right)^2 - (d_1d_2 - s^2)} \\ &= \frac{d_1 + d_2}{2} - \sqrt{\left(\frac{d_1 - d_2}{2}\right)^2 + s^2}, \end{aligned}$$

where the last step uses $(d_1 + d_2)^2/4 - d_1 d_2 = (d_1 - d_2)^2/4$. The hypothesis therefore becomes

$$\frac{d_1 + d_2}{2} - |a| > \sqrt{\left(\frac{d_1 - d_2}{2}\right)^2 + s^2}. \quad (\text{A.1})$$

The right-hand side is a norm and the left-hand side is greater than it, so it is positive too. Before squaring, we record a bound on $|a|$ that we will need below: dropping s^2 under the root in (A.1) only decreases the right-hand side, so $\sqrt{\left(\frac{d_1 - d_2}{2}\right)^2 + s^2} \geq \frac{|d_1 - d_2|}{2}$ and therefore

$$\frac{d_1 + d_2}{2} - |a| > \frac{|d_1 - d_2|}{2} \implies |a| < \frac{(d_1 + d_2) - |d_1 - d_2|}{2} = \min(d_1, d_2) \leq \sqrt{d_1 d_2}. \quad (\text{A.2})$$

Squaring (A.1), and using $(d_1 + d_2)^2 - (d_1 - d_2)^2 = 4d_1 d_2$ and rearranging, it becomes

$$d_1 d_2 - (d_1 + d_2)|a| + a^2 > s^2. \quad (\text{A.3})$$

By AM-GM, $d_1 + d_2 \geq 2\sqrt{d_1 d_2}$, so the left-hand side of (A.3) only decreases when $d_1 + d_2$ is replaced by $2\sqrt{d_1 d_2}$:

$$\begin{aligned} \left(\sqrt{d_1 d_2} - |a|\right)^2 &= d_1 d_2 - 2\sqrt{d_1 d_2}|a| + a^2 \\ &\geq d_1 d_2 - (d_1 + d_2)|a| + a^2 \\ &> s^2. \end{aligned}$$

By (A.2) we have $\sqrt{d_1 d_2} > |a|$, so we may take positive square roots to obtain $\sqrt{d_1 d_2} - |a| > |s|$, that is, $\sqrt{d_1 d_2} > |s| + |a|$. Therefore

$$\begin{aligned} |c^+ c^-| &= |s^2 - a^2| \\ &= |s - a||s + a| \\ &\leq (|s| + |a|)^2 \\ &< d_1 d_2, \end{aligned}$$

which gives $\rho(J^f)^2 = |c^+ c^-|/(d_1 d_2) < 1$, as required. $\square$

When the entries are blocks rather than scalars, the argument breaks down. The product $c^+ c^-$ becomes a product of matrices which is non-commuting, so it no longer reduces to the single number $s^2 - a^2$ and $\rho(J^f)$ is no longer controlled by it directly; the AM-GM step has no matrix analogue valid in this generality; and there is no closed-form for $\lambda_{\min}(\text{Sym}(H^f))$ to turn the hypothesis into a usable scalar inequality. This is why the general conjecture remains open and why in section 4.2.1 we settle instead for the stronger per-edge dominance condition of theorem 5.

A few computer simulations indicate that this condition is sufficient, and in fact it seems to be considerably stricter than necessary.

APPENDIX B. DECLARATION OF USE OF GENERATIVE AI

This work's use of generative AI was in accordance with the University's guidelines in <https://uc3m.libguides.com/en/MFP/GenAI>. Per said guidelines, what follows is a form detailing the author's (Odysseas Machairas) personal use of generative AI (henceforth in first person).

DECLARATION OF USE OF GENERATIVE ARTIFICIAL INTELLIGENCE (AI) IN MASTER FINAL PROJECT (TFM)

I have used Generative AI in my TFM

Check all that apply:

YES	NO
-----	----

If you have ticked YES, please complete the following 3 parts of this document:

Part 1: Reflection on legal, ethical and responsible behavior

Please be aware that the use of AI carries some risks and may lead to a series of serious academic consequences: the TFM will not be evaluated by the University if the use of AI involves the use of confidential data, copyrighted materials, or personal data, used without complying with the conditions required in each case (authorization of the interested parties, authorization of the owners, following the instructions of the University).

Question	
1. In my interactions with AI tools, I have provided confidential data, always with the appropriate authorization of the data subjects. Confidentiality encompasses any information that a person or organization wishes to protect for legal, commercial, privacy, or strategic reasons (such as patents or trade secrets).	
YES, I have used this data with permission	NO, I have not used confidential data
2. In my interaction with Generative AI tools, I have submitted copyrighted materials with the permission of those concerned.	
YES, I have used these materials with permission from the copyright holders, or without permission because they fall within one of the exceptions or limitations permitted by law: • Public domain work • Licensed work (Creative Commons licenses) • Use of excerpts for research purposes (fair use)	NO, I have not used copyrighted materials

3. In my interaction with Generative AI tools, I have submitted personal data with the consent of the data subjects.	
YES, I have used this data with the authorization of the interested parties and in accordance with the instructions contained in the guide approved by the University .	NO, I have not used personal data
4. My use of the Generative AI tool has respected its terms of use , as well as the essential ethical principles, not being maliciously oriented to obtain an inappropriate result for the work presented, that is to say, one that produces an impression or knowledge contrary to the reality of the results obtained, that supplants my own work or that could harm people.	
YES	NO

Part 2: Declaration of technical use

Use the following model statement as many times as necessary, in order to reflect all types of iteration you have had with Generative AI tools. Include one example for each type of use where indicated: *[Add an example]*.

I declare that I have made use of the Generative AI system Claude Sonnet 4.6 and Claude Opus 4.8 for:

Documentation and drafting:

- *Supporting reflection in relation to the development of the work: iterative process of analysing alternatives and approaches using AI*

I had an approach (forced systems via Galley's idea) that had an example that seemed to imply that the approach would go nowhere; I asked the AI whether it saw something I missed or if my reasoning was wrong.

I asked the AI about common extensions of the notion of a positive definite matrix for non-symmetric matrices (the forced systems part has two approaches that came from this)

- *Revision or rewriting of previously drafted paragraphs*

I asked a few times to revise paragraphs with some notation that I wasn't sure if it was standard or not (I expect the AI to have enough training data to point out if I'm writing something particularly weird). I asked to check for informalisms (such as contractions) and ideas to rewrite or move particularly awkward paragraphs. E.g.:

- "Does this need more explanation, or would it be understood by any reader in applied math?"
- "Do you think these paragraphs are good and better help the reader understand, or should I change them/remove them? I want to motivate Theorem 5 but it seems like I'm doing it in a bit of a roundabout way."

- *Search for information or answers to specific questions*

I used AI often as a sanity-check to see if there is something obviously wrong in a line of reasoning, mostly with differential geometry (again, as a sanity-check, not a source). E.g.:

- "How is the pullback defined for $SO(3)$ using $\mathbb{R}^3$ as vectors and covectors? I'm getting that the pullback of a covector α at a point g , call it ξ , must satisfy that for all tangent vectors η , $\text{trace}(\text{skew}(\xi), \text{skew}(\eta).T) = \text{trace}(\text{skew}(\alpha), \text{skew}(g @ \eta).T)$. Is this correct?"
- "How accurate is it to say that the reason why we can work in the Lie algebra (i.e., we can left trivialize) is that the group structure encodes some notion of symmetry which makes all tangent spaces structurally the same?"

- *Bibliography search*

I used AI to find some linear algebra theorems that I could use for proving convergence in forced systems. Namely, it brought up Gelfand's formula and Weyl's inequality. I also used it to surface a book that speaks in general about geometric integration (E. Hairer, G. Wanner, and C. Lubich, *Geometric Numerical Integration*, 8th entry in bibliography)

- *Summary of bibliography consulted* **Not used**
- *Translation of texts consulted* **Not used**

Develop specific content

Generative AI has been used as a support tool for the development of the specific content of the TFM, including:

- *Assistance in the development of lines of code (programming)*

I used it to get common ways to achieve certain API patterns in Python that I'm familiar from other languages (namely, Rust). E.g.:

- "What is the most ergonomic way to be able to either give a continuous lagrangian/force and discretize it automatically, or give directly the discretized version, and make both still count as a 'Scenario'? I feel like I almost want to make scenarios inherit Scenario and ContinuousLagrangian at the same time?"
- "Is there a way to communicate that jax arrays are immutable? I get a runtime error when importing if I don't do `field(default_factory=)` but it is very annoying to repeat, and I think it's genuinely not necessary for correctness in the case of jax arrays."

- *Generation of diagrams, images, audios or videos*

I used AI to get basic scaffolding for Tikz diagrams, and for small snippets (like how to show two markers and a type of line in one line in a matplotlib legend).

I asked how to add some interactivity to notebooks that I used to explore examples (this is not directly included in the paper, but it was used to make iterating on the figures easier).

I used AI to get somewhat reasonable starting points for parameters used in the examples (although they ended up being heavily changed in almost all cases).

- *Optimisation processes **Not used***
- *Data processing: collection, analysis, cross-checking of data... **Not used***
- *Inspiration of ideas in the creative process **Not used***
- *Other uses linked to the generation of specific points of the specific development of the paper*

I made a script to run and graph all examples automatically using the code in the interactive notebooks, and I asked the AI for a few things like how to import code with `uv` from a different directory. I also asked it to resolve some LaTeX build errors.

If you think it is necessary to include it, add it in each case:

using the prompt ...

"I'm going to sleep. Meanwhile, I want you to do a thorough review of what I have so far in the thesis. I want you to do 2-3 passes, noting down places where something could be improved. I will want to change @body/6-conclusions.tex, so don't pay too much attention there.

General things I want you to keep in mind:

- *Is the content clear? Does it naturally flow from the previous part?*
 - *Is every topic properly introduced? Is something out of order?*
 - *Is it 100% mathematically rigorous? Are analogies and explanations rigorous too?*
- But also use your discretion to find more things wrong with it. Please go ahead."*

having as interaction ...

Doing a "fresh" bird's eye review of what I have written so far, as a double check that nothing is horribly wrong, and to consider what sections I should prioritize to make nicer, if any (of course, this was just secondary criteria after my and my supervisor's judgement)

Part 3: Reflection on utility

Please provide a personal assessment (free format) of the strengths and weaknesses you have identified in the use of Generative AI tools in the development of your work. Mention if it has helped you in the learning process, or in the development or drawing conclusions from your work.

Generally, I try to limit my use of AI and I try to have good “hygiene” to make sure I don’t get *skill atrophy* (see, e.g., <https://news.harvard.edu/gazette/story/2025/11/is-ai-dulling-our-minds/>). That is, I try to think about answers myself and not reach for AI as the first thing. However, I think it can be useful for tasks that are the correct mix of generic and/or unimportant. My use of AI can be split into two main categories.

The first was to bridge the background I have with the topics that I wanted to study. I have a weird background in the sense that I have barely studied mathematics formally, but I have always been really interested. That gets reflected in that I am comfortable with abstraction and formality but I have little practical experience. Most introductory materials implicitly assume the opposite, so there is a cognitive mismatch. Here, AI is very useful because it can very specifically answer questions the way I want to hear them. It’s not that I *replaced* books with AI, I just used it to see if I was understanding it correctly or to see what I’m missing in a step that didn’t make sense to me.

As a specific example, I didn’t get why a the derivative of a function $f : Q \rightarrow R^n$ resulted in a covector, but the AI pointed out that it’s just because the tangent space of R^n is R^n . I could have tried to look it up online, but often for these more elementary questions it requires more expertise to find *where* the answer is than just finding the answer yourself directly. I could also have asked my supervisor, but for these small things the AI was quicker and it allowed me to discuss more interesting topics with my supervisor that are a better use of both of our times.

Indeed, the AI did not replace my supervisor at all, and many things it cannot answer well. These are things that require more “taste”, that are a bit more convoluted (e.g., the AI could explain how to discretize but struggled to say *why*, and when I discussed it with my supervisor it became clear that it was actually surprisingly subtle); as well as general guidance (which is arguably the most important part of the supervisor).

In the theoretical side, it was also useful for finding useful linear algebra theorems. My supervisor told me an anecdote of having various linear algebra books that mostly list a bunch of theorems because they might be useful, and I think for these cases the pattern matching machine that is AI is particularly useful and well-employed. Most of the theorems I believe I could have proved with other methods, but the proofs would have probably been way less concise and harder to understand and verify.

The second part it was useful was for things where I want to “get something out”. In this sense, I used AI as “advanced rubber-ducking”, where “rubber-ducking” is the technique of explaining a problem to a rubber duck which can be helpful because verbalizing the problem helps you understand it and potentially solve it. When I didn’t know how to start a paragraph, I sometimes asked it to give me some way to start it. I’m very particular, so almost always I didn’t really like the recommendation, but that made it way easier to help me understand what I *do* want to say, and then it was simple to write it myself from scratch.

I thought that writing plotting code and giving me example parameters would be one of these “unimportant and mechanical” use cases, but I found out that I always wanted something slightly different than what it gave me, and it tended to clutter and complicate both code and plots, and after a couple of attempts I found out it was significantly easier for myself to write all code. Even for the examples, I had to make them myself to get interesting representative behavior that is easy to see in static images, which the AI was remarkably unsuccessful at. The most useful

place of AI in this sense was to help me write the Lagrangians for rotational systems, since (a) I have never properly studied moments of inertia, (b) it was not the point of the paper, and (c) it is something that is ultimately pretty elementary.

Overall, there were legitimate uses of AI for this thesis, especially for unblocking myself in both theoretical understanding and writing, but I tried not to rely on it too much and I tried to use it towards the lower end of what is expected in this day and age.